# Midterm Exam Review

# Fixed-Point
# vs.
# Floating-Point

# Unsigned vs. Signed Numbers in Binary Number System (BNS)

- 2's complement is standard signed representation in fixed-point (FXP) format
- sign-magnitude is used in floating-point (FLP) format
- biased is used for **exponent** in FLP
- e.g. 4-bit integer (with radix-point to the right)

| $x_3x_2x_1x_0$ | unsigned | signed | | | |
|---|---|---|---|---|---|
| | | 2's complement | 1's complement | sign-magnitude | biased |
| | $2^3 2^2 2^1 2^0$ | $-X \Rightarrow -X+16$<br>$\overline{x_3}\overline{x_2}\overline{x_1}\overline{x_0}+0001$<br>$(-2^3)2^2 2^1 2^0$ | $-X \Rightarrow -X+15$<br>$\overline{x_3}\overline{x_2}\overline{x_1}\overline{x_0}$ | $\pm 2^2 2^1 2^0$<br>$-X = \overline{x_3}x_2x_1x_0$ | unsigned - 7 |
| range | [0, 15] | [-8, 7] | [-7, 7] | [-7, 7] | [-7, 8] |
| 0000 | 0 | 0 | 0 | 0 | -7 |
| 0001 | 1 | 1 | 1 | 1 | -6 |
| 0010 | 2 | 2 | 2 | 2 | -5 |
| 0011 | 3 | 3 | 3 | 3 | -4 |
| 0100 | 4 | 4 | 4 | 4 | -3 |
| 0101 | 5 | 5 | 5 | 5 | -2 |
| 0110 | 6 | 6 | 6 | 6 | -1 |
| 0111 | 7 | 7 | 7 | 7 | 0 |
| 1000 | 8 | -8 | -7 | -0 | 1 |
| 1001 | 9 | -7 | -6 | -1 | 2 |
| 1010 | 10 | -6 | -5 | -2 | 3 |
| 1011 | 11 | -5 | -4 | -3 | 4 |
| 1100 | 12 | -4 | -3 | -4 | 5 |
| 1101 | 13 | -3 | -2 | -5 | 6 |
| 1110 | 14 | -2 | -1 | -6 | 7 |
| 1111 | 15 | -1 | -0 | -7 | 8 |




2's complement signed

$Y = y_3y_2y_1y_0$

$-Y = \overline{y_3}\overline{y_2}\overline{y_1}\overline{y_0} + 0001$

$X - Y = X + (-Y)$

```
   0  1  1  1 (positive:  7)
+  1  1  0  1 (negative:  -3)
-------------
   0  1  0  0 (positive: +4)
```

normal

```
   0  1  1  1 (postive: +7)
+  0  0  0  1 (positive: +1)
-------------
   1  0  0  0 (negative: -8)
```

overflow!

```
   1  1  1  1 (negative:  -1)
+  1  0  0  0 (negative:  -8)
-------------
   0  1  1  1 (positive: +7)
```

underflow!

| $x_3x_2x_1x_0$ | unsigned | signed | | | |
|---|---|---|---|---|---|
| | | 2's complement | 1's complement | sign-magnitude | biased |
| | $2^3 2^2 2^1 2^0$ | $-X \Rightarrow -X+16$<br>$\overline{x_3x_2x_1x_0}+0001$<br>$(-2^3)2^2 2^1 2^0$ | $-X \Rightarrow -X+15$<br>$\overline{x_3x_2x_1x_0}$ | $\pm 2^2 2^1 2^0$<br>$-X = \overline{x_3}x_2x_1x_0$ | unsigned - 7 |
| range | [0, 15] | [-8, 7] | [-7, 7] | [-7, 7] | [-7, 8] |
| 0000 | 0 | 0 | 0 | 0 | -7 |
| 0001 | 1 | 1 | 1 | 1 | -6 |
| 0010 | 2 | 2 | 2 | 2 | -5 |
| 0011 | 3 | 3 | 3 | 3 | -4 |
| 0100 | 4 | 4 | 4 | 4 | -3 |
| 0101 | 5 | 5 | 5 | 5 | -2 |
| 0110 | 6 | 6 | 6 | 6 | -1 |
| 0111 | 7 | 7 | 7 | 7 | 0 |
| 1000 | 8 | -8 | -7 | -0 | 1 |
| 1001 | 9 | -7 | -6 | -1 | 2 |
| 1010 | 10 | -6 | -5 | -2 | 3 |
| 1011 | 11 | -5 | -4 | -3 | 4 |
| 1100 | 12 | -4 | -3 | -4 | 5 |
| 1101 | 13 | -3 | -2 | -5 | 6 |
| 1110 | 14 | -2 | -1 | -6 | 7 |
| 1111 | 15 | -1 | -0 | -7 | 8 |

# Fixed-Point Numbers

Positional representation: *m* whole (integer) and *n* fractional digits (format **Q*m.n***, say Q32.0, or Q16.16, or Q1.31)

Value of a number: $x = (x_{m-1}x_{m-2}\ldots x_1x_0 \cdot x_{-1}x_{-2}\ldots x_{-n})_2 = \sum x_i\, 2^i$

For example:

$$2.375 = (10.011)_{two} = (1\times2^1) + (0\times2^0) + (0\times2^{-1}) + (1\times2^{-2}) + (1\times2^{-3})$$

**unsigned** numbers in the **range [0, $2^m - ulp$]** representable, where ***ulp* = $2^{-n}$**

**signed** numbers in the **range [$-2^{m-1}$, $2^{m-1} - ulp$]** representable, where ***ulp* = $2^{-n}$**

*ulp*: unit in least (significant) position

**Fixed-point** (e.g., Q16.16) arithmetic is same as **integer** (e.g., Q32.0)

**signed** two's complement properties (including sign change) hold here as well:

$(01.011)_{2's\text{-}compl} = \mathbf{(-0\times2^1)} + (1\times2^0) + (0\times2^{-1}) + (1\times2^{-2}) + (1\times2^{-3}) = +1.375$

$(11.011)_{2's\text{-}compl} = \mathbf{(-1\times2^1)} + (1\times2^0) + (0\times2^{-1}) + (1\times2^{-2}) + (1\times2^{-3}) = -0.625$

# Floating-Point Numbers

Useful for applications where very large and very small numbers are needed simultaneously

- Fixed-point representation must sacrifice precision for small values to represent large values

  $x = (0000\ 0000\ .\ 0000\ 1001)_{two}$ Small number

  $y = (1001\ 0000\ .\ 0000\ 0000)_{two}$ Large number

- Neither $y^2$ nor $y / x$ is representable in the format above

- Floating-point representation is like scientific notation:

  $-20\ 000\ 000 = -2 \times 10^7$ $0.000\ 000\ 007 = 7 \times 10^{-9}$

Significand

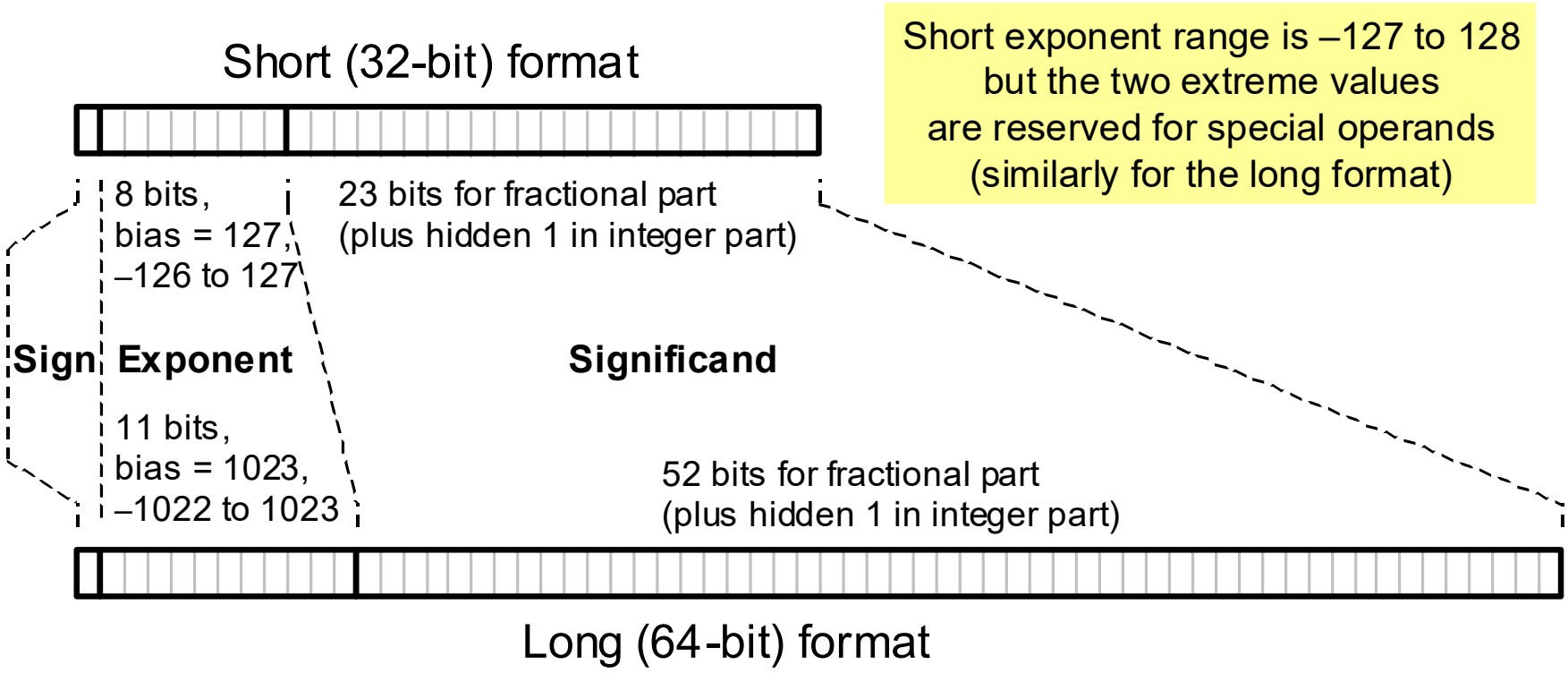
Exponent base

Exponent

Also, 7E–9

# ANSI/IEEE Standard Floating-Point Format (IEEE 754)

Revision (IEEE 754R) is being considered by a committee




Short exponent range is –127 to 128
but the two extreme values
are reserved for special operands
(similarly for the long format)

The two ANSI/IEEE standard floating-point formats.

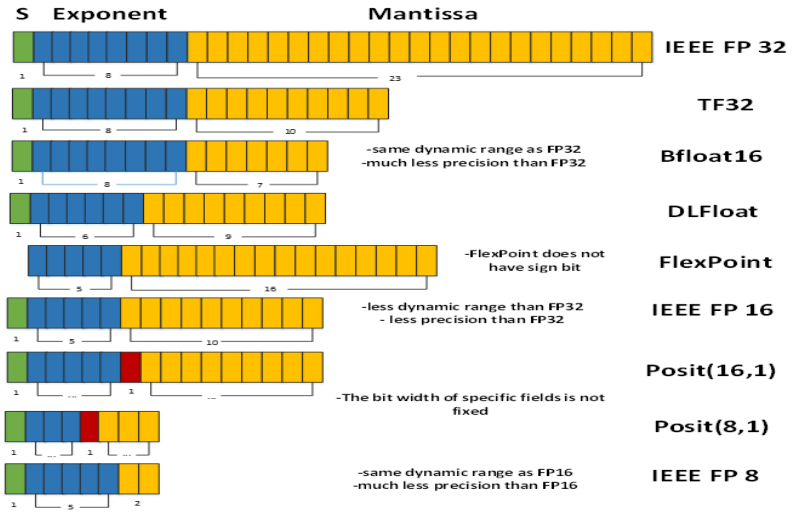
# Short and Long IEEE 754 Formats: Features

| Feature | Single/Short | Double/Long |
|---|---|---|
| Word width in bits | 32 | 64 |
| Significand in bits | 23 + 1 hidden | 52 + 1 hidden |
| Significand range | $[1, 2 - 2^{-23}]$ | $[1, 2 - 2^{-52}]$ |
| Exponent bits | 8 | 11 |
| Exponent bias | 127 | 1023 |
| Zero ($\pm 0$) | $e$ + bias = 0, $f$ = 0 | $e$ + bias = 0, $f$ = 0 |
| Denormal | $e$ + bias = 0, $f \neq 0$<br>represents $\pm 0.f \times 2^{-126}$ | $e$ + bias = 0, $f \neq 0$<br>represents $\pm 0.f \times 2^{-1022}$ |
| Infinity ($\pm\infty$) | $e$ + bias = 255, $f$ = 0 | $e$ + bias = 2047, $f$ = 0 |
| Not-a-number (NaN) | $e$ + bias = 255, $f \neq 0$ | $e$ + bias = 2047, $f \neq 0$ |
| Ordinary number | $e$ + bias $\in [1, 254]$<br>$e \in [-126, 127]$<br>represents $1.f \times 2^e$ | $e$ + bias $\in [1, 2046]$<br>$e \in [-1022, 1023]$<br>represents $1.f \times 2^e$ |
| *min* | $2^{-126} \cong 1.2 \times 10^{-38}$ | $2^{-1022} \cong 2.2 \times 10^{-308}$ |
| *max* | $\cong 2^{128} \cong 3.4 \times 10^{38}$ | $\cong 2^{1024} \cong 1.8 \times 10^{308}$ |

**significand**: s=1.f
**mantissa**: m= .f (fractional part of significand)

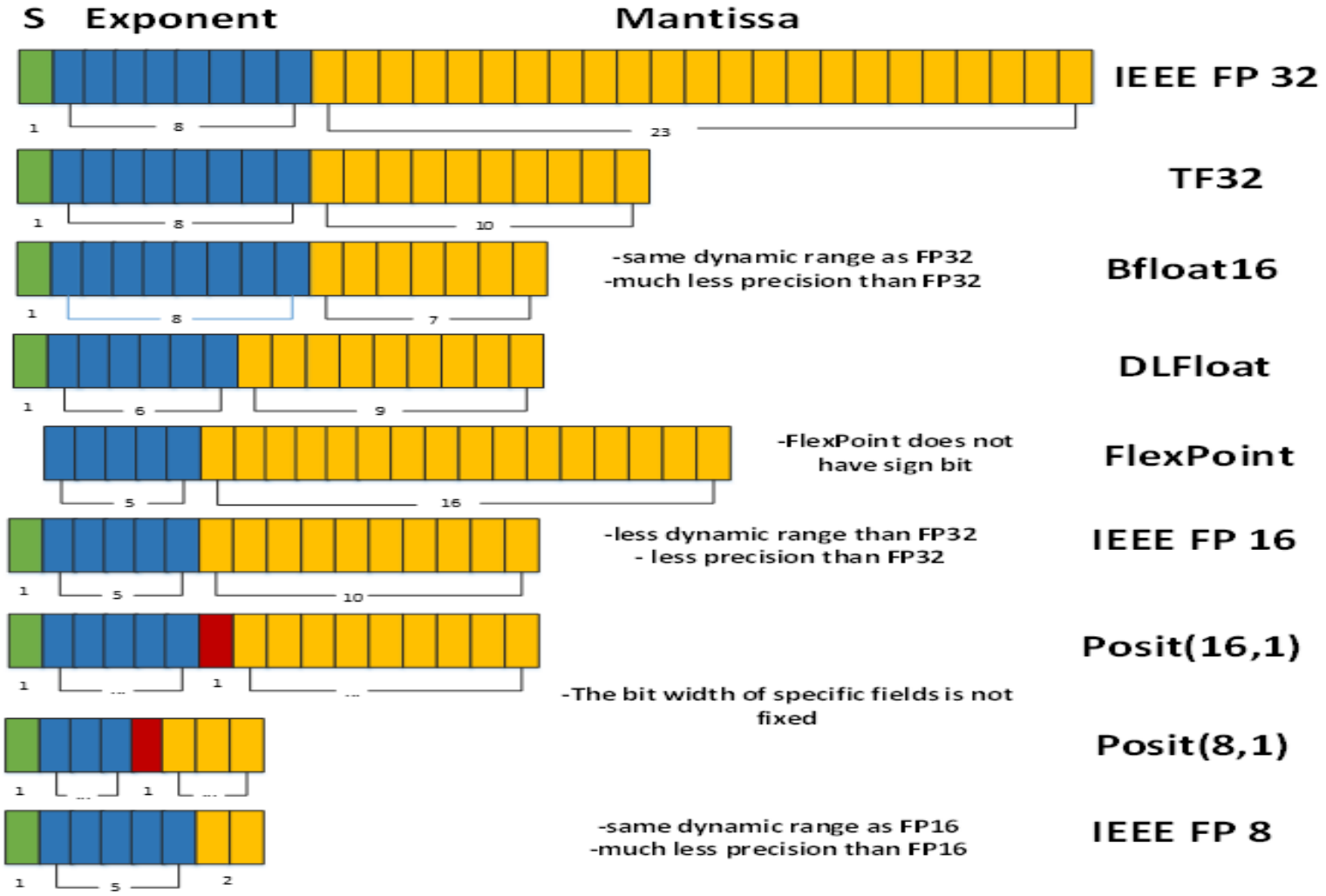
# Fixed-Point (FXP) vs. Floating-Point (FLP)

- FXP: Qm.n (with m integer bits and n fractional bits)
  - range: $2^m$
  - ***absolute*** precision (unit of last place, or ulp): $2^{-n}$ (e.g., $2^{-16}$ in Q16.16)
- FLP: 1+exponent € + mantissa (m) (e.g., 1+8+23 for FLP32)
  - range: $2^{2\text{^}exp}$
  - ***relative*** precision: $2^{-mantissa}$ (e.g., $2^{-23}$ in $1.11\ldots1*2^{100}$, or $1.11\ldots1*2^{-100}$ in FLP32)
- representable range
  - exponent in FLP
  - integer bits in FXP
  - FLP >> FXP (e.g., $2^{31}$ vs. $2^{128}$ for Q32.0 vs. FLP32 in 32-bit formats)
- precision
  - mantissa in FLP
  - fractional bits in FXP
- complexity
  - FLP >> FXP (more area and delay for FLP)
- machine learning
  - training
    - 32-b, 16-b or 8-b FLP
  - inference
    - 16-b or 8-b FXP



# Some FLP-like Formats in ML

- in machine learning application



# Quantization

- quantization
  - determine proper bit-width in the hardware arithmetic component from the almost full-precision representation in software (such as double floating-point format in C)
- bit-width affects area, speed, and power
- fixed-point is (FXP) versus floating-point (FLP)
- bit-width is related to range and/or precision
- in FXP, 16-bit Q12.4 has larger range than 8-bit Q4.4, while Q4.12 has higher precision than Q4.4
  - but hardware adder for Q12.4 and for Q4.12 is the same
  - designer should know the position of fractional point
- in FLP, maximum exponent determine range while # of mantissa bits determines *relative* precision

# FLP Arithmetic

# Overview of IEEE 754 Standard Formats

Some features of the ANSI/IEEE standard floating-point number representation formats.

| Feature | Single / Short | Double / Long |
|---|---|---|
| Word width (bits) | 32 | 64 |
| Significand bits | 23 + 1 hidden | 52 + 1 hidden |
| Significand range | $[1, 2 - 2^{-23}]$ | $[1, 2 - 2^{-52}]$ |
| Exponent bits | 8 | 11 |
| Exponent bias | 127 | 1023 |
| Zero ($\pm 0$) | $e + bias = 0$, $f = 0$ | $e + bias = 0$, $f = 0$ |
| Denormal | $e + bias = 0$, $f \neq 0$<br>represents $\pm 0.f \times 2^{-126}$ | $e + bias = 0$, $f \neq 0$<br>represents $\pm 0.f \times 2^{-1022}$ |
| Infinity ($\pm\infty$) | $e + bias = 255$, $f = 0$ | $e + bias = 2047$, $f = 0$ |
| Not-a-number (NaN) | $e + bias = 255$, $f \neq 0$ | $e + bias = 2047$, $f \neq 0$ |
| Ordinary number | $e + bias \in [1, 254]$<br>$e \in [-126, 127]$<br>represents $1.f \times 2^e$ | $e + bias \in [1, 2046]$<br>$e \in [-1022, 1023]$<br>represents $1.f \times 2^e$ |
| *min* | $2^{-126} \cong 1.2 \times 10^{-38}$ | $2^{-1022} \cong 2.2 \times 10^{-308}$ |
| *max* | $\cong 2^{128} \cong 3.4 \times 10^{38}$ | $\cong 2^{1024} \cong 1.8 \times 10^{308}$ |

# Ordinary and Special Numbers Summary

- **Reserve exponent = 0 and 255 for special numbers**

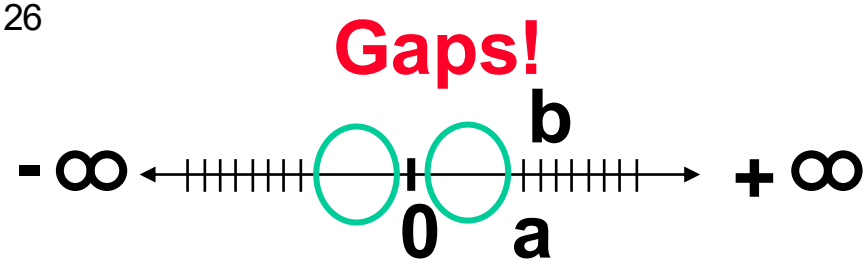
| Exponent (8-bit) | Significand (23-bit) | meaning |
|---|---|---|
| **0** | **0** | **0** |
| **0** | **nonzero** | **Denorm** |
| 1-254 | anything | +/- ordinary numbers |
| **255** | **0** | **+/- ∞** |
| **255** | **nonzero** | **NaN** |

# Denorm (Subnorm)

- There's a gap among representable numbers near 0
  - Smallest representable positive number:
    $a = 1.000\ldots\ldots000_2 * 2^{-126} = 2^{-126}$
  - Second smallest representable positive number:
    $b = 1.000\ldots\ldots001_2 * 2^{-126}$
    $= (1 + 0.000\ldots\ldots001_2) * 2^{-126}$
    $= (1 + 2^{-23}) * 2^{-126}$
    $= 2^{-126} + 2^{-149}$

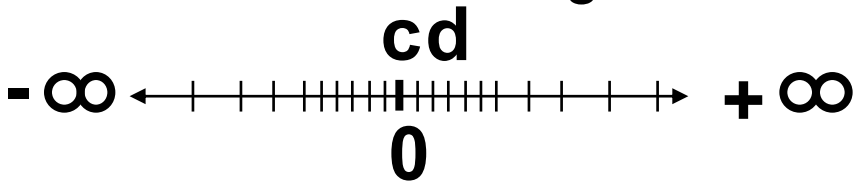
  $a - 0 = 2^{-126}$

  $b - a = 2^{-149}$



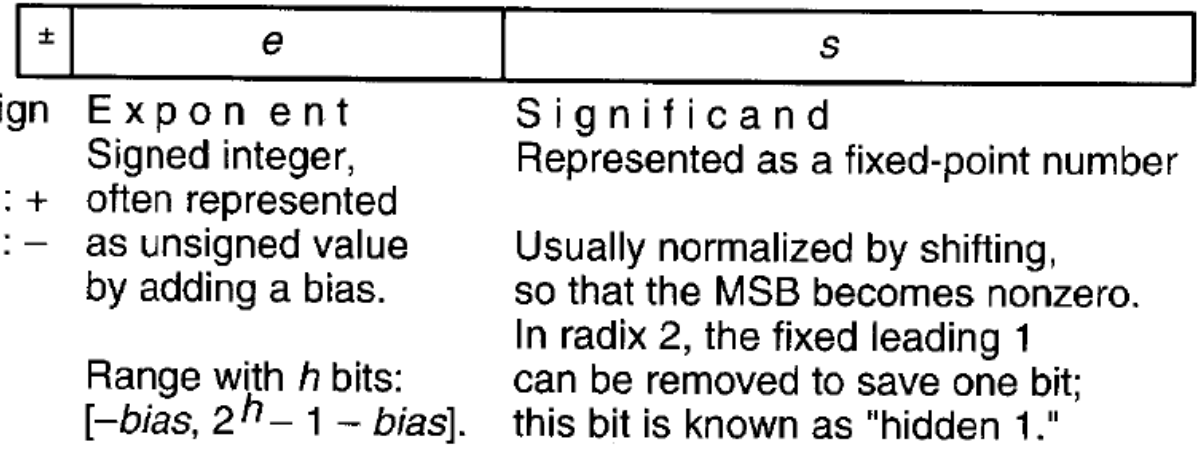

**due to normalization and implicit 1**

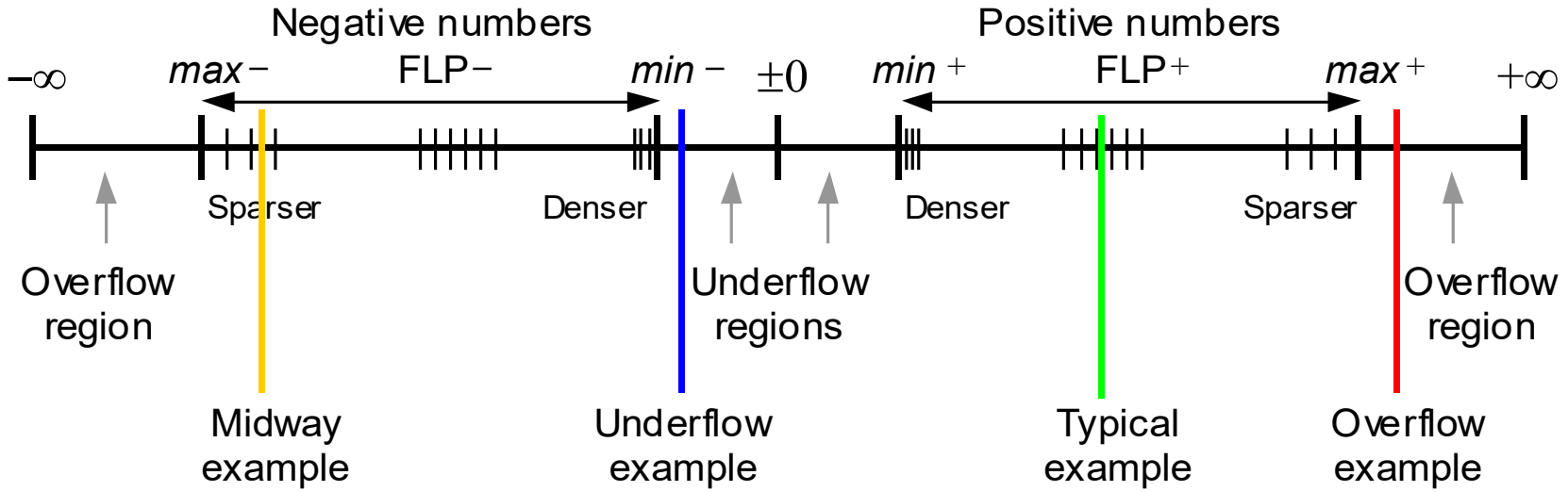
- we have not yet used exponent=0 and non-zero significand
  - **De-norm**alized numbers:
    - ✓ no (implicated leading 1
    - ✓ implicit exponent = -126
    - ✓ smallest positive number: $c = 0.00\ldots001 * 2^{-126} = 2^{-149}$
    - ✓ second smallest positive number: $d = 0.00\ldots010 * 2^{-126} = 2^{-148}$
    - ✓ distance of two neighboring subnorms: $2^{-149}$



# Floating-Point Number Format and Distribution

unlike fixed-point (FXP) which has constant distance between two neighboring numbers, FLP has **variable** distance between two neighboring numbers



# Requirements for FLP Arithmetic (Normalization)

significand of operation result s must be 1<=s <2
finding leading 1/0 and then shift

**Example:**

$(1 + 2^{-1} + 2^{-23}) \quad - \quad (1 + 2^{-1})$

result $2^{-23} = 0.000\ldots01$

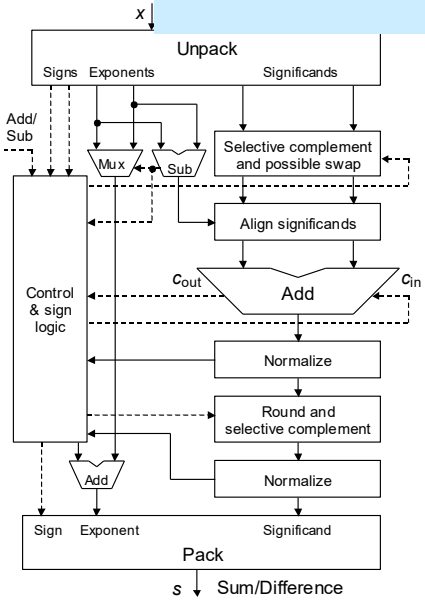
normalized result 1 (with exponent – 23)

# Basic Floating-Point Algorithms

**Addition:** $x + y = (\pm s1 \times b^{e1}) + (\pm s2 \times b^{e2})$

Assume $e1 \geq e2$; *alignment* ***right*** *shift* (*pre-shift*) of y is needed if $e1 > e2$

$$(\pm s1 \times b^{e1}) + (\pm s2 \times b^{e2}) = (\pm s1 \times b^{e1}) + (\pm s2 / b^{e1-e2}) \times b^{e1}$$
$$= (\pm s1 \pm s2 / b^{e1-e2}) \times b^{e1} = \pm s \times b^{e}$$




**Example:** Numbers to be added:

$$x = 2^5 \times 1.00101101$$
$$y = 2^1 \times 1.11101101 \leftarrow$$

Operand with smaller exponent to be preshifted

Operands after alignment shift:

$$x = 2^5 \times 1.00101101$$
$$y = 2^5 \times 0.000111101101$$

Result of addition:

$$s = 2^5 \times 1.010010111101$$
$$s = 2^5 \times 1.01001100 \leftarrow$$

Extra bits to be rounded off

Rounded sum

**(1)Pre-alignment;**
**(2)Fixed-point arithmetic;**
**(3) Post-normalization;**
**(4) Rounding**

# normalization for add/sub (two cases)

- **Case-I (Far Case): abs(e1-e2) >= 2,**
  - **normalization of shift at most one bit**

```
      1.10011111
      0.00101011
ADD   ----------
      1.11001010
```

Exponent difference is large (3 in this case)
No normalization is needed

$$1 \le s_1 < 2,\ 1 \le s_2 < 2$$

$$1/4 \le s_2 \times 2^{-2} < 1/2$$

$$\frac{1}{2} \le s_1 \pm s_2 \times 2^{-2} < 2\frac{1}{2}$$

```
      1.1001111
      0.0110110
ADD   ---------
     10.0000101
NORM  1.00000101
```

Significand is larger than 2
normalization of right shift 1-bit is needed

Quiz: Give an example that normalization requires 1-bit left shift

- **Case-II (Near Case): abs(e1-e2) <=1,**
  - **normalization of arbitrary shift bits**

$$1 \le s_1 < 2,\ 1 \le s_2 < 2$$

$$1/2 \le s_2 \times 2^{-1} < 1,$$

$$0 < s_1 \pm s_2 \times 2^{-1} < 3$$

$$-1 < s_1 \pm s_2 < 4$$

```
      1.1001111
      1.1001010
SUB   ---------
      0.0000101
NORM  1.0100000
```

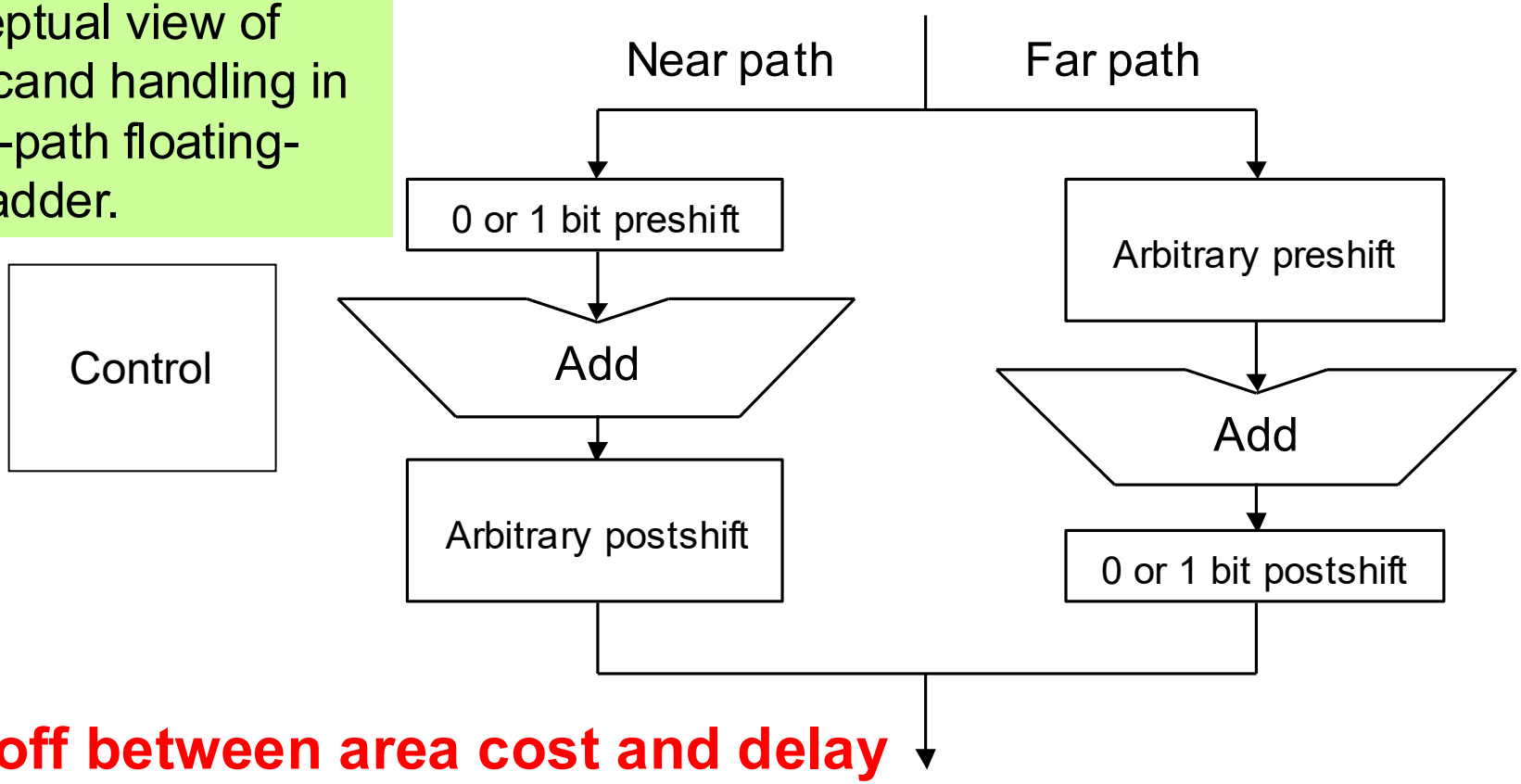
Significand has many leading zeros
normalization of left shift many bits is needed

# Floating-Point Adder with Dual Data Paths

Amount of alignment right-shift

One bit: Arbitrary left shift may be needed due to cancellation
Two bits or more: Normalization left-shift will be by at most one bit

Conceptual view of significand handling in a dual-path floating-point adder.

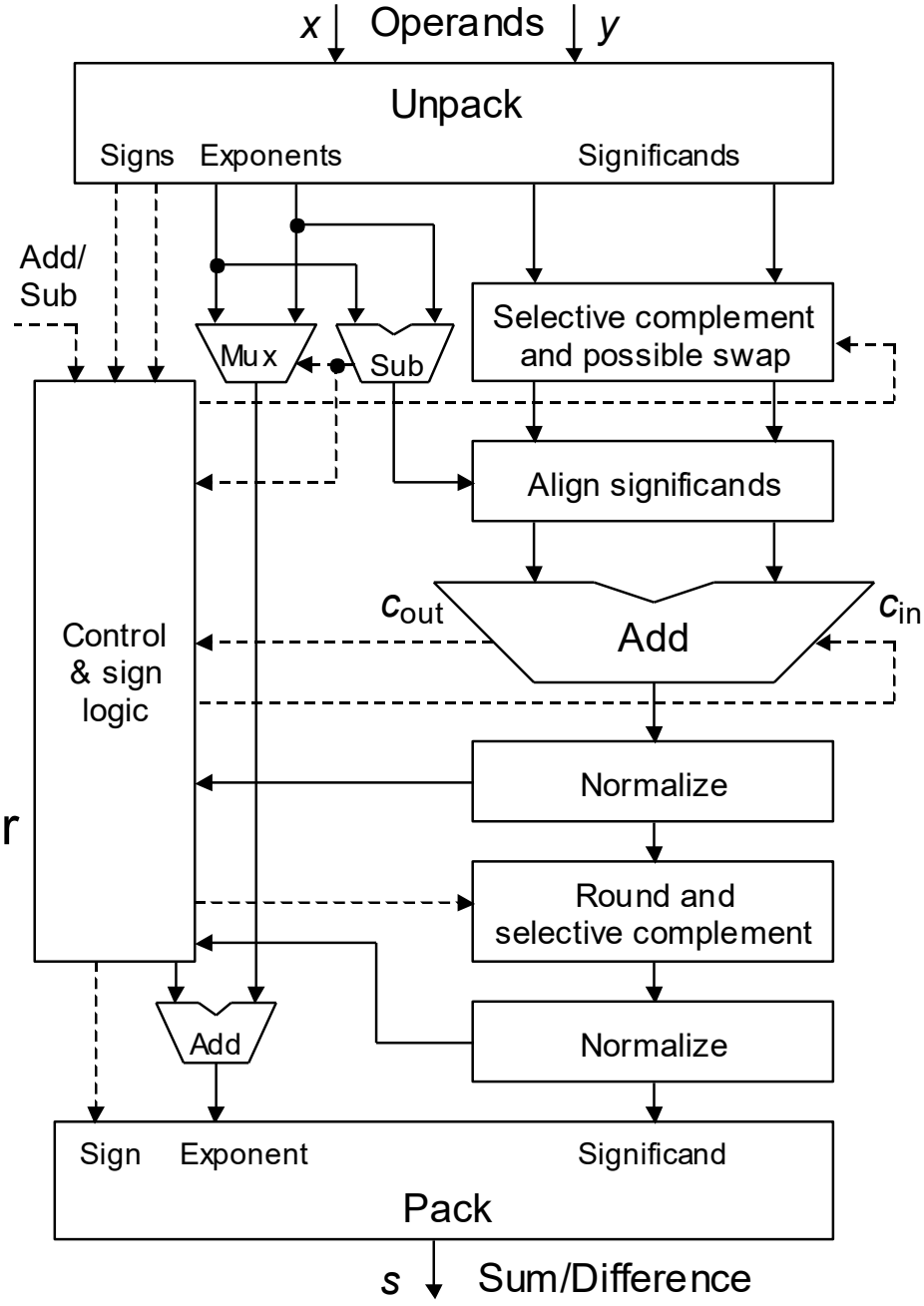



**trade-off between area cost and delay ($T_{sh}+T_{add}$ vs. $2*T_{sh}+T_{add}$)!**

# Basic algorithm for FLP add/sub

- **Determine exception**
  - Special numbers
- **Subtract exponents**
- **Align significands**
  - Right-shift
- **Add (subtract)**
  - Fixed-point arithmetic
- **Normalize**
  - Leading one detector, and shifter
- **Round**

$$T = T_{unpack} + \underbrace{T_{sub} + T_{sh}}_{T_{alignment}} + T_{add} + \underbrace{T_{LZD} + T_{sh}}_{T_{normalization}} + T_{rnd} + T_{pack}$$



# alignment only for -25 <= $e_a - e_b$ <= 25

- denote (exponent, signed mantissa) as
  - A: ($e_a$, $m_a$), B: ($e_b$, $m_b$)
  - $|m_a| < 2$ ($-2 < m_a \leq 1$, or $1 \leq m_a < 2$), similarly for $m_b$
  - $m_a$, $m_b$ could be represented as Q2.23
- assume $e_a > e_b$, exponent difference $ed = e_a - e_b$
  - If ed >= 26, and alignment-right-shifted $m_b$ -> $m'_b$
  - $|m'_b| < 2^{-25}$ (because $|m_b| < 2$)
  - after possible 1-bit normalization left shift, the impact on add result is $< 2^{-24}$ , which could be ignored after rounding
- similarly for $e_a < e_b$
- thus, could consider only -25 <= $e_a - e_b$ <= 25
  - alignment with ed = 26 or more could be ignored
- Note for the possible 1-bit left shift normalization of add result, which could amplify error by a factor of 2

# Floating-Point Multiplication and Division

## Multiplication

$$(\pm s1 \times b^{e1}) \times (\pm s2 \times b^{e2}) = (\pm s1 \times s2) \times b^{e1+e2}$$

$s1 \times s2 \in [1, 4)$, post-shifting (**right**-shift) may be needed for normalization

Overflow or underflow can occur during multiplication or normalization

## Division

$$(\pm s1 \times b^{e1}) / (\pm s2 \times b^{e2}) = (\pm s1 / s2) \times b^{e1-e2}$$

$s1 / s2 \in (0.5, 2)$, post-shifting (**left** shift) may be needed for normalization

Overflow or underflow can occur during division or normalization

# Floating-Point Multipliers

$$(\pm s1 \times b^{e1}) \times (\pm s2 \times b^{e2}) = (\pm s1 \times s2) \times b^{e1+e2}$$

$s1 \times s2 \in [1, 4)$: may need postshifting

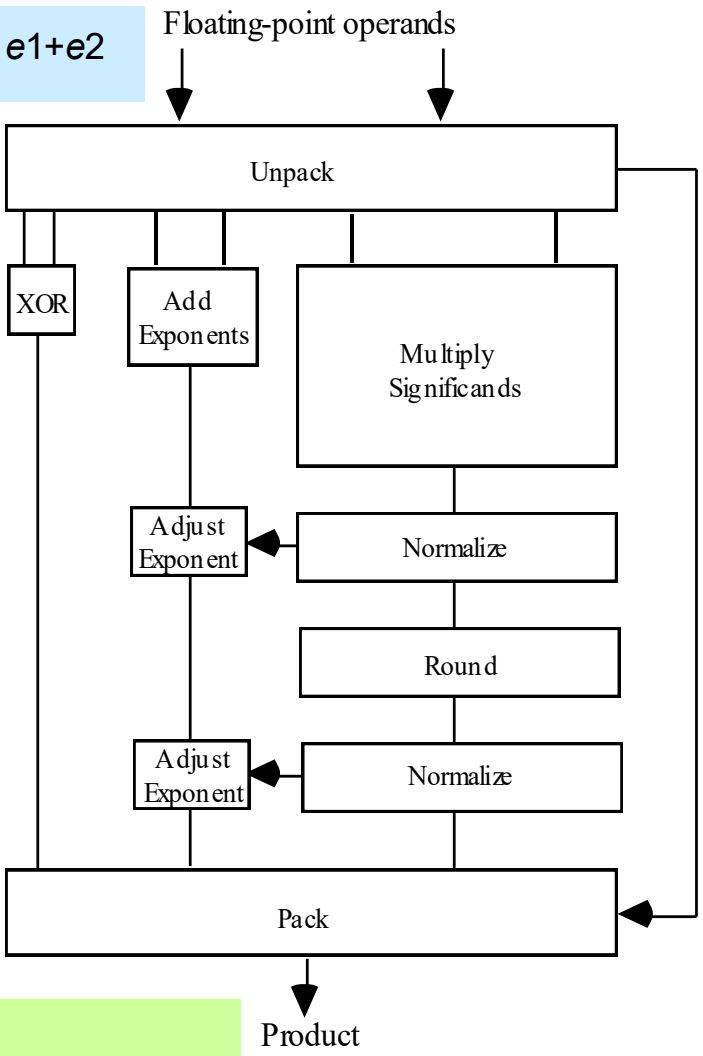
Overflow or underflow can occur during multiplication or normalization

**Speed considerations**

Many multipliers produce the lower half of the product (rounding info) early

Need for normalizing right-shift is known at or near the end

Hence, rounding can be integrated in the generation of the upper half, by producing two versions of these bits




Block diagram of a floating-point multiplier.

# Error Analysis in FLP Arithmetic

# Sources of Computational Errors

FLP approximates exact computation with real numbers

Two sources of errors to understand and counteract:

- Representation errors

  e.g., no ***exact*** machine representation for $0.1_{10}=0.000110011001100\ldots_2$, 1/3, $\sqrt{2}$, or $\pi$

- Arithmetic errors

  e.g., $(1 + 2^{-12})^2 = 1 + 2^{-11} + 2^{-24}$
  not representable in IEEE single-precision format due to the term $2^{-24}$

errors due to finite precision can lead to disasters in life-critical applications

error format:
***absolute*** error = $(x_{comp} - x^*)$ , usually used in fixed-point (FXP)
***relative*** error = $(x_{comp} - x^*) / x^*$, usually used in floating-point (FLP)

$$\text{relative error } e = \frac{x_{comp} - x^*}{x^*} \quad \text{Þ} \quad x_{comp} = (1+e)x^*$$

# Error FXP  add and multiply

- C = A + B with absolution representation error eabs for both A and B
  - C = (A*+eabs) + (B*+ eabs) = (A*+B*) + 2xeabs
  - Fxp add error for C = eabs + eabs = 2*eabs
- C = A x B = (A*+eabs) x (B*+ eabs)
  = A* x B* + (A*+B*)xeabs + eabs x eabs
  - Fxp multiuply for C = (A*+B*)xeabs + eabs x eabs

# Error Analysis for FP Multiplication and Division

## Errors in floating-point multiplication

$x_{fp} = (1 + \sigma)x$
$\sigma$ is the relative representation error of x

$y_{fp} = (1 + \tau)x$
$\tau$ is the relative representation error of y

Consider the positive operands $x_{fp}$ and $y_{fp}$

$$\begin{aligned} x_{fp} \times_{fp} y_{fp} &= (1 + \eta)\, x_{fp} y_{fp} \\ &= (1 + \eta)(1 + \sigma)(1 + \tau)\, xy \\ &= (1 + \eta + \sigma + \tau + \eta\sigma + \eta\tau + \sigma\tau + \eta\sigma\tau)\, xy \\ &\cong (1 + \eta + \sigma + \tau)\, xy \end{aligned}$$

$\tau$ is the relative arithmetic error of $x_{fp} \times_{fp} y_{fp}$

## Errors in floating-point division

Again, consider positive operands $x_{fp}$ and $y_{fp}$

$$\begin{aligned} x_{fp} /_{fp} y_{fp} &= (1 + \eta)\, x_{fp} / y_{fp} \\ &= (1 + \eta)(1 + \sigma)x / [(1 + \tau)y] \\ &= (1 + \eta)(1 + \sigma)(1 - \tau)(1 + \tau^2)(1 + \tau^4)(\ldots)\, x/y \\ &\cong (1 + \eta + \sigma - \tau)\, x/y \end{aligned}$$

# Error Analysis for FP Addition and Subtraction

## Errors in floating-point addition

Consider the positive operands $x_{fp}$ and $y_{fp}$

$$
\begin{aligned}
x_{fp} +_{fp} y_{fp} &= (1 + \eta)(x_{fp} + y_{fp}) \\
&= (1 + \eta)(x + \sigma x + y + \tau y) \\
&= (1 + \eta)(1 + \frac{\sigma x + \tau y}{x + y})(x + y)
\end{aligned}
$$

Magnitude of this ratio is upper-bounded by $max(|\sigma|, |\tau|)$, so the overall error is no more than $|\eta| + max(|\sigma|, |\tau|)$

## Errors in floating-point subtraction

Again, consider positive operands $x_{fp}$ and $y_{fp}$

$$
\begin{aligned}
x_{fp} -_{fp} y_{fp} &= (1 + \eta)(x_{fp} - y_{fp}) \\
&= (1 + \eta)(x + \sigma x - y - \tau y) \\
&= (1 + \eta)(1 + \frac{\sigma x - \tau y}{x - y})(x - y)
\end{aligned}
$$

This term also unbounded for subtraction without any guard bits

Magnitude of this ratio can be very large if $x$ and $y$ are both large but $x - y$ is relatively small (recall that $\tau$ can be negative)

called *cancellation or loss of significance*

# Worst-Case Error Accumulation

In a sequence of operations, round-off errors might add up

The larger the number of cascaded computation steps (that depend on results from previous steps), the greater the chance for, and the magnitude of, accumulated errors

With rounding, errors of opposite signs tend to cancel each other out in the long run, but one cannot count on such cancellations

**Practical implications:**

Perform intermediate computations with a higher precision than what is required in the final result

Implement multiply-accumulate in hardware (DSP chips)

Reduce the number of cascaded arithmetic operations; So, using computationally more efficient algorithms has the double benefit of reducing the execution time as well as accumulated errors

# Range of Multiplication

- 64-bit product of two 32-bit signed **integers** x, y
  - if we only allow 32-bit product, which part should we take?

$-2^{31} \le x, y < 2^{31} - 1 \quad \Rightarrow -2^{62} + 2^{31} < x * y \le 2^{62}$

$00\times\times\cdots\times$

$01\times\times\cdots\times$

$11\times\times\cdots\times$

$10\times\times\cdots\times < -2^{62}$ (impossible)

- 64-bit product of two 32-bit signed **fractional** x, y
  - if we only allow 32-bit product, which part should we take?

$-1 \le x, y < 1 \quad \Rightarrow -1 < x * y \le 1$

$00.\times\times\cdots\times$

$01.\times\times\cdots\times$

$11.\times\times\cdots\times$

$10.\times\times\cdots\times < -1$ (impossible)

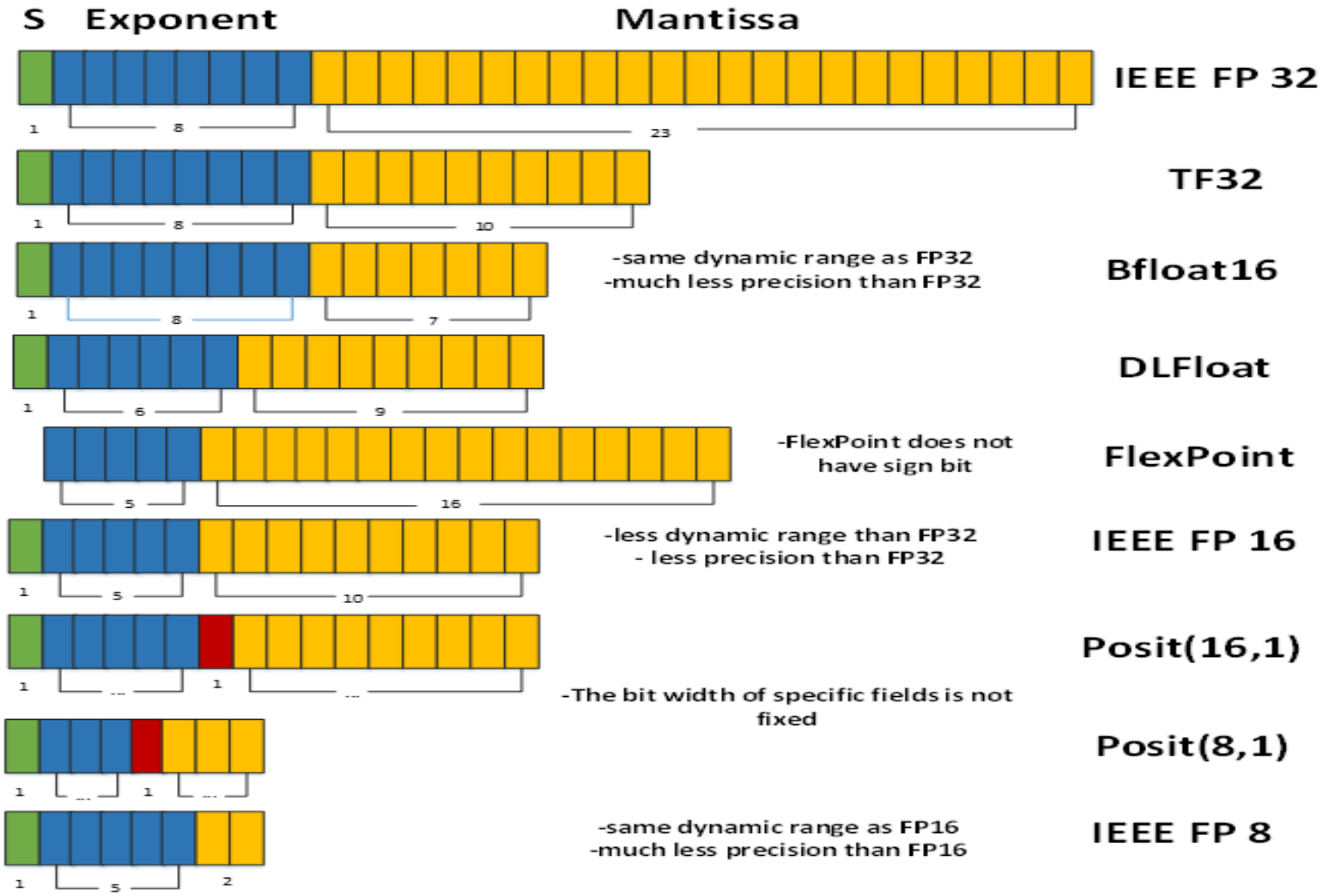
# Range of Many Multiply-Add

- worse-case range
  - accumulation of all positive (or negative) products
  - e.g., if magnitude of all 1024 products is smaller than 1
  - 10 MSB guard bits are required
- simulation to obtain the range of the final results
  - neglect the rarely appeared outliers, say 3 sigma assuming normal distribution
    - ✓ using **saturation** to clip the outliers to max/min representable numbers
  - smaller number of guard bits

# Posit Number

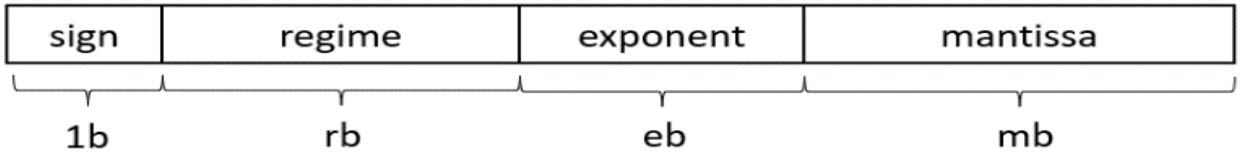
# Other FLP-like Representation

- IEEE FLP-32 (1+8+23), IEEE FLP-16 (1+5+10)
- Intel Bfloat (1+8+7), IBM DLfloat (1+6+9), Nvidia TF32 (1+8+10)

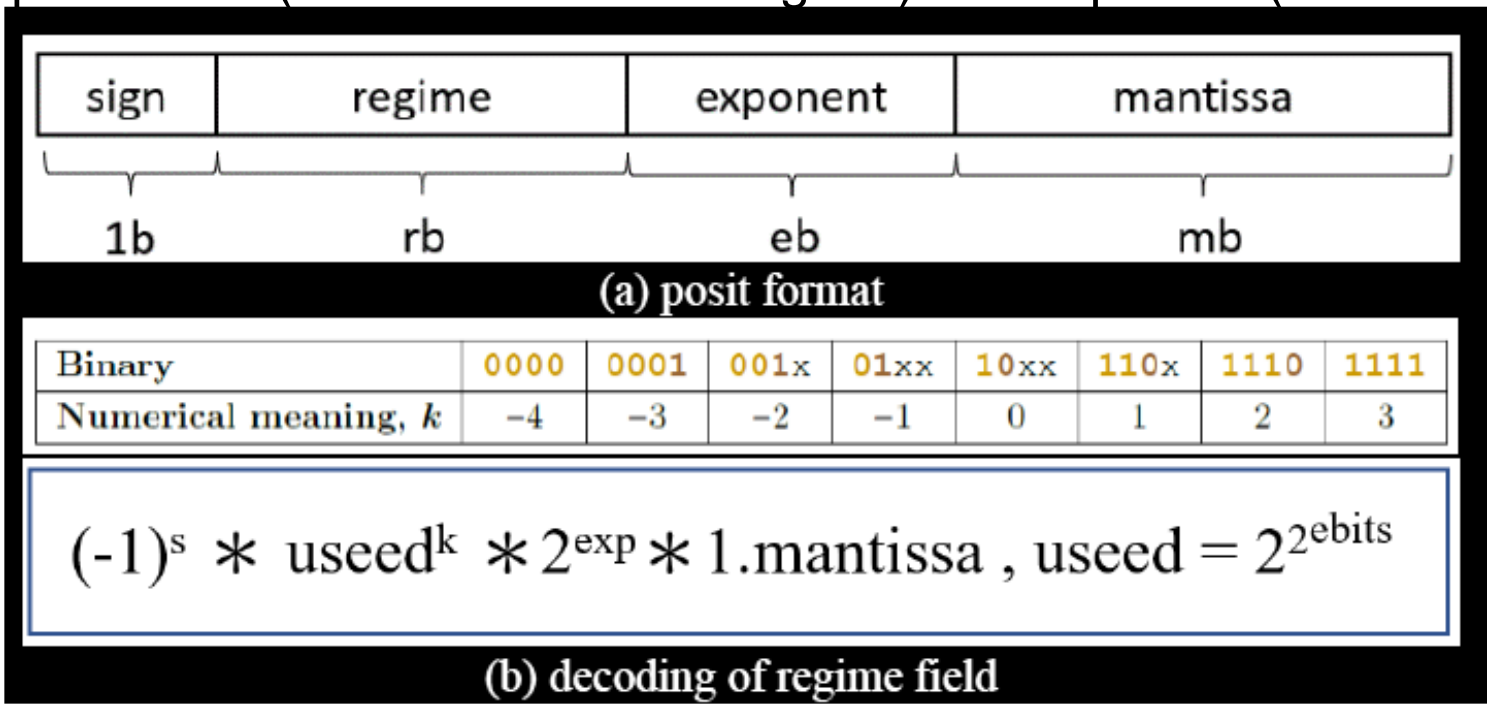


# Posit (2017)

- dynamically changeable exponent field and mantissa field
- sign (1 b) + regime (>=2 b) + exponent (given eb)+ fractional (>=0 b)




- total exponent = k (from bit-inverse in regime) and exponent (from ebits)




$$11101011 \xrightarrow{2'com} 00010101 \xrightarrow{RLC} \underset{S\ R\ E\ M}{0_001_01_01} \longrightarrow -(2^4)^{-2} * 2^1 * (1+2^{-2})$$

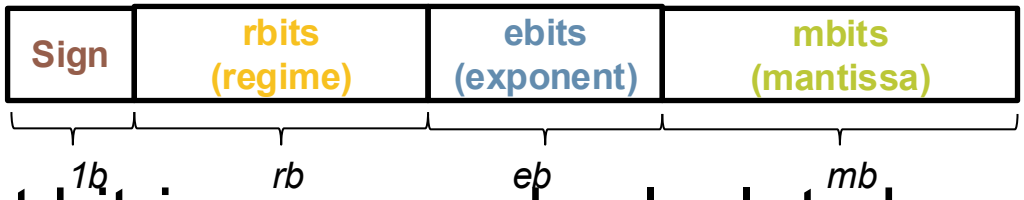
# Range (IEEE FLP vs. Posit)

- 16-b posit with ebits=1 has larger representation range than 16-b IEEE FLP
- 8-b posit is mostly enough for machine learning applications
- classification inference (prediction) usually use FXP, but training of neural network models usually requires FLP format due to larger dynamic range

| Size, Bits | IEEE Float Exp. Size | Approx. IEEE Float Dynamic Range | Posit ebits | Approx. Posit Dynamic Range |
|---|---|---|---|---|
| 16 | 5 | $6 \times 10^{-8}$ to $7 \times 10^{4}$ | 1 | $4 \times 10^{-9}$ to $3 \times 10^{8}$ |
| 32 | 8 | $1 \times 10^{-45}$ to $3 \times 10^{38}$ | 3 | $6 \times 10^{-73}$ to $2 \times 10^{72}$ |
| 64 | 11 | $5 \times 10^{-324}$ to $2 \times 10^{308}$ | 4 | $2 \times 10^{-299}$ to $4 \times 10^{298}$ |
| 128 | 15 | $6 \times 10^{-4966}$ to $1 \times 10^{4932}$ | 7 | $1 \times 10^{-4855}$ to $1 \times 10^{4855}$ |
| 256 | 19 | $2 \times 10^{-78984}$ to $2 \times 10^{78913}$ | 10 | $2 \times 10^{-78297}$ to $5 \times 10^{78296}$ |

# Posit Format

- sign, rbits (regime), ebits (exp), mbits (fractional)




- find first bit-inverse and calculate k
  - negative k = # of leading zeros
    - ✓ if rbits = 000…0 => k = -(n-1-eb), mb=0
  - positive k = # of leading ones – 1
    - ✓ if rbits = 111…11 => k = (n-1-eb)-1 , mb=0
  - 2 <= rb <= n-1–eb
    - ✓ excluding sign and ebits
- total_exp = k*(2^eb) + ebits

| Binary | 0000 | 0001 | 001x | 01xx | 10xx | 110x | 1110 | 1111 |
|---|---|---|---|---|---|---|---|---|
| Numerical meaning, $k$ | −4 | −3 | −2 | −1 | 0 | 1 | 2 | 3 |

$$(-1)^{s} * \text{useed}^{k} * 2^{\text{exp}} * 1.\text{mantissa} \text{ , useed} = 2^{2^{\text{ebits}}}$$

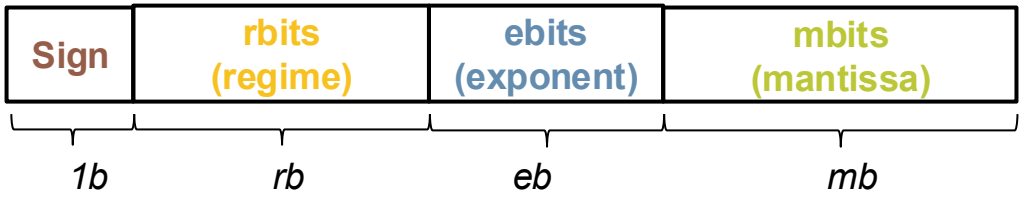
# Posit Decoding/Encoding

- Posit decoding (e.g., 8-bit 00011011, eb=1)
  - 2's complement if s=1 (s=0 => 0_0011011)
  - leading bit-inverse for k (0_001_1_011, k = -2, ebits=1)
    - ✓# leading zeros for negative k, e.g., 001x… => k= -2 )
    - ✓# of leading ones – 1 for positive k (e.g., 110x… => k=2-1=1)
  - total_exp from k, eb, ebits ( total_exp = (-2)*2^1 + 1)
    - ✓(2^eb * k) + ebits
  - mantissa from mbits (mantissa=1.mbits = 1.011)
    - ✓mbits are fractional bits
- Posit encoding (e.g., 1.011 * 2^1101, eb=1)
  - total_exp => k, ebits ( k= -2 (110), ebits=1 (1) )
  - k => rbits (regime (k= -2 => rbits=001 => x_001_1_xxx )
  - mantissa=1.mbits => mbits (1.mbits=1.011 => x_001_1_011)
    - ✓rounding if necessry
  - 2's complement if s=1 (s=0 => 0_001_1_011)

# Posit Decoding

| Binary | 0000 | 0001 | 001x | 01xx | 10xx | 110x | 1110 | 1111 |
|---|---|---|---|---|---|---|---|---|
| Numerical meaning, $k$ | −4 | −3 | −2 | −1 | 0 | 1 | 2 | 3 |

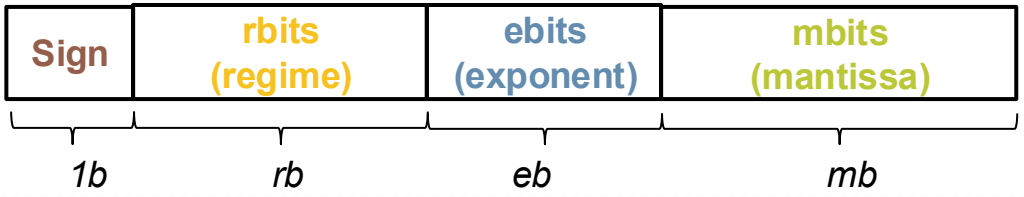
$$(-1)^s * useed^k * 2^{exp} * 1.mantissa\ ,\ useed = 2^{2^{ebits}}$$




- Extract total exponent and mantissa
  - input: posit format with given ebits
  - output: sign, total exponent, mantissa

$$(-1)^s * useed^k * 2^{exponent} * 1.mantissa\ ,\ useed=2^{2^{eb}}$$

# Regime




- rb is not fixed
  - 2 <= rb <= n-1-eb  for n-bit posit with given eb
- total exponent = $2^{eb} * k + ebits$
  - (k, ebits) are quotient and remainder when total exponent is divided by $2^{eb}$
  - k is minus # of leading 0's in 0-string before 1,
  - or k is # of leading 1's in the 1-string before 0 minus 1
- if mb=0, mantissa = 1

| n=8, eb=2 regime (mbits) | k | mb |
|---|---|---|
| 00000 | -5 | 0 |
| 00001 | -4 | 0 |
| 0001 | -3 | 1 |
| 001 | -2 | 2 |
| 01 | -1 | 3 |
| 10 | 0 | 3 |
| 110 | 1 | 2 |
| 1110 | 2 | 1 |
| 11110 | 3 | 0 |
| 11111 | 4 | 0 |
| | | |

| Binary | 0000 | 0001 | 001x | 01xx | 10xx | 110x | 1110 | 111 |
|---|---|---|---|---|---|---|---|---|
| Numerical meaning, $k$ | −4 | −3 | −2 | −1 | 0 | 1 | 2 | 3 |

$$(-1)^s * useed^k * 2^{exp} * 1.mantissa\ ,\ useed = 2^{2^{ebits}}$$

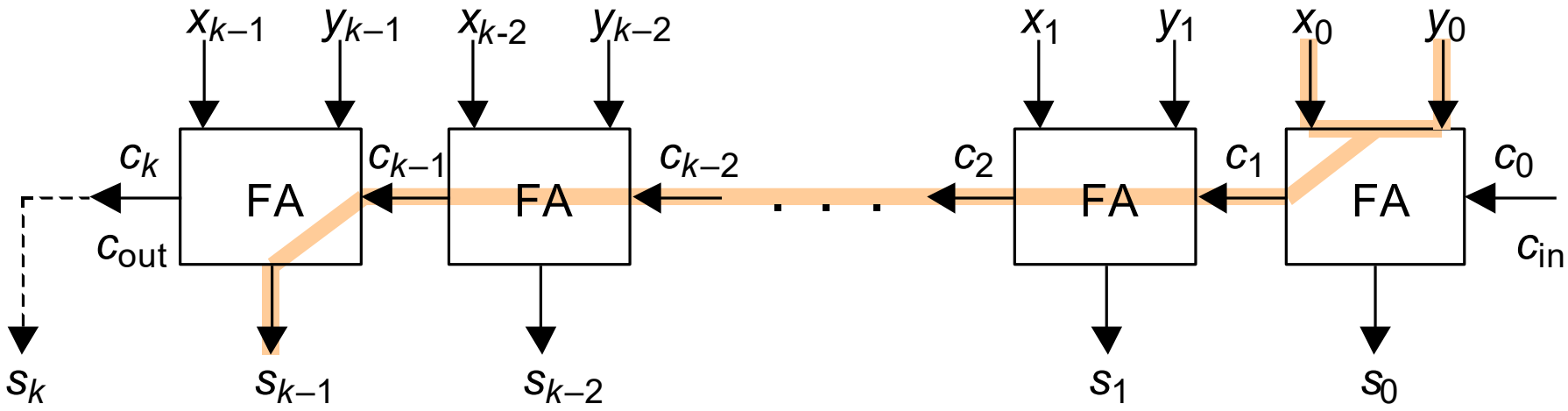
# Carry Save Addition (CSA)

# Ripple-Carry Adder (RCA)

**The simplest implementation of carry propagate adder (CPA) which takes two inputs and generate one ouput**

**assume T(xy->s) > T(xy->cout) > T(cin->s) > T(cin->cout)**

$$T_{\text{ripple-add}} = T_{FA}(x,y \rightarrow c_{out}) + (k-2) \times T_{FA}(c_{in} \rightarrow cout) + T_{FA}(c_{in} \rightarrow s)$$




Critical path in a *k*-bit ripple-carry adder.
Delay is O(k) for k-bit precision

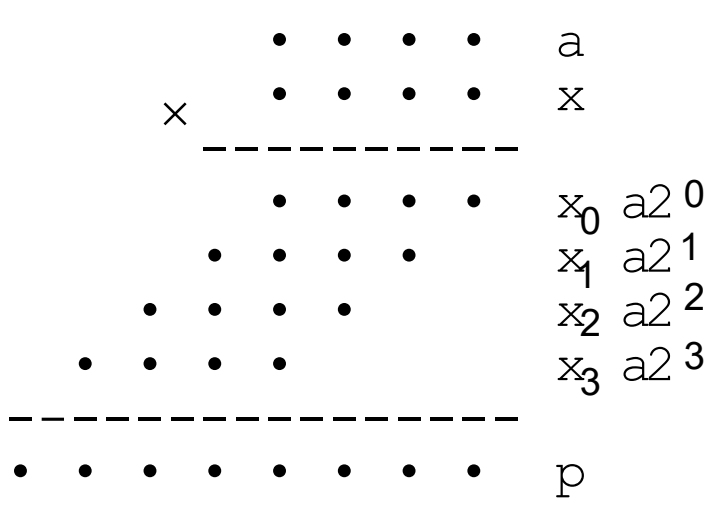
# CPA vs. CSA

- k-bit carry propagate adder (CPA) takes **two** inputs of k-bit and generate **one** output
  - ripple carry adder (RCA): delay **O(k)**
  - carry look-ahead adder (CLA): delay **O(logk)**
  - carry select adder: delay O(k~ logk)
  - carry skip adder: delay O(k ~ logk)
  - conditional sum adder: delay O(logk)
  - parallel prefix adder: delay O(logk)
  - hybrid of the above
    - ✓ design trade-off between delay and area
  - carry-limited adder: delay **O(1)**
    - ✓ in redundant number system, not in binary number system
- carry save adder (CSA) takes **multiple (>=3)** inputs and generate **two** outputs

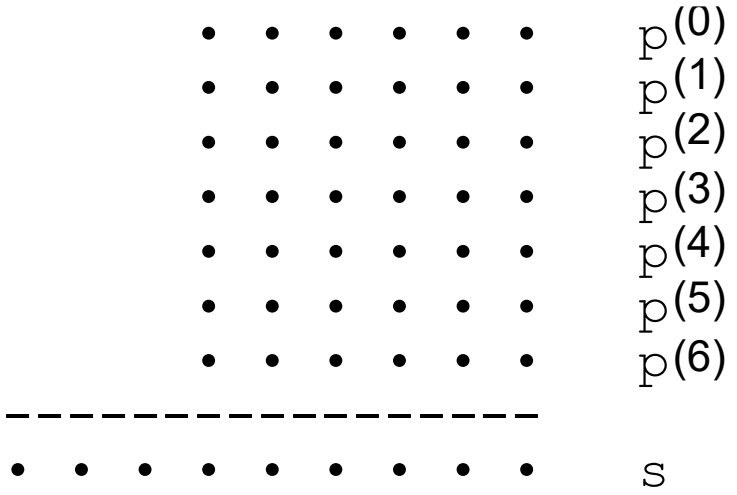
# Addition of several input operands

Some applications of multioperand addition

## 4-bit x 4-bit multiplier

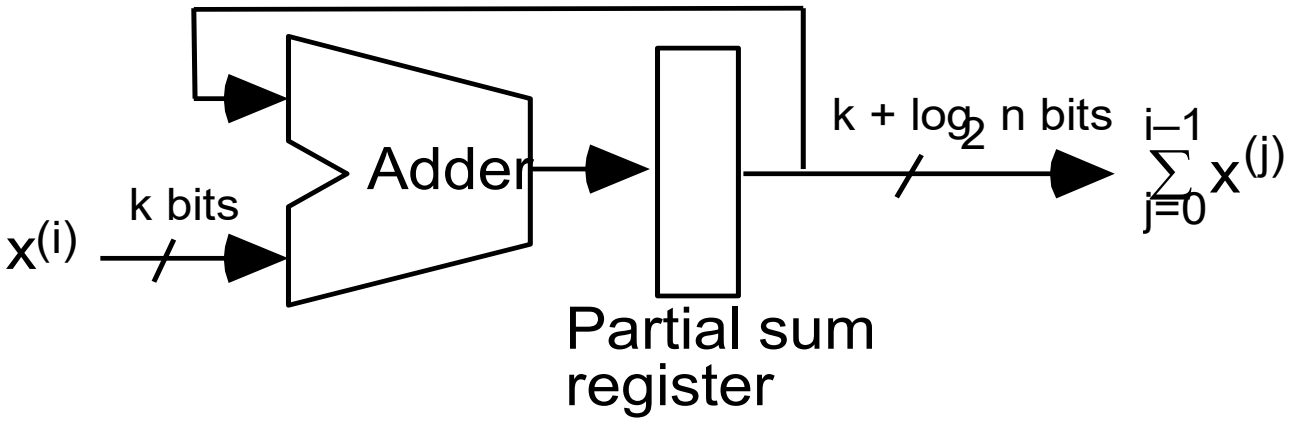



## 7-input adder




Multioperand addition problems for multiplication or inner-product computation in dot notation.

# Serial Implementation with One Adder




Serial implementation of multi-operand addition with a single 2-operand adder.

Delay of k-bit fast adder: O(logk)

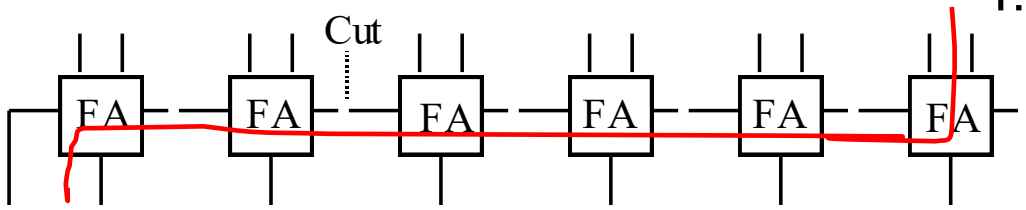
$$T_{\text{serial-multi-add}} = O(n \log(k + \log n))$$
$$= O(n \log k + n \log \log n)$$

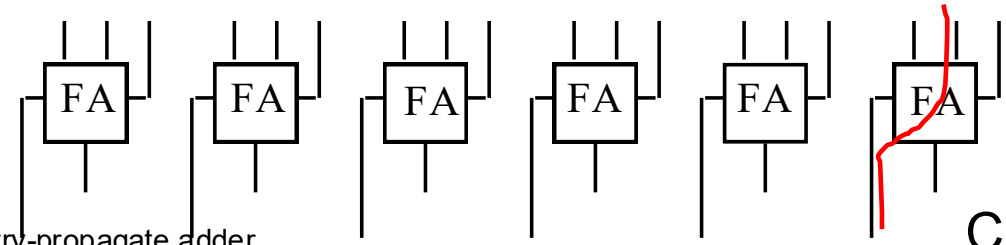
Therefore, addition time grows super-linearly with $n$ when $k$ is fixed and logarithmically with $k$ for a given $n$

# Carry-Save Adders (CSA)

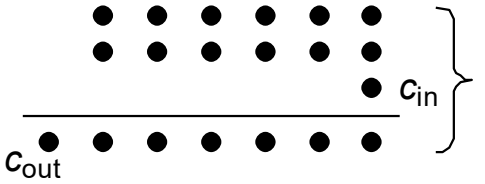
RCA
T: O(k)

A ripple-carry adder turns into a carry-save adder if the carries are saved (stored) rather than propagated.

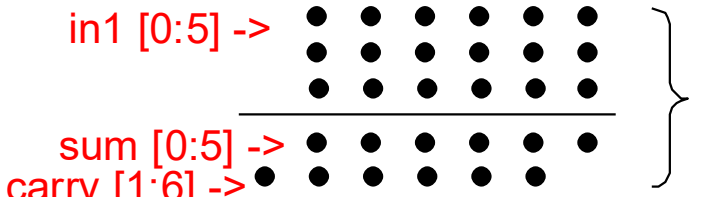


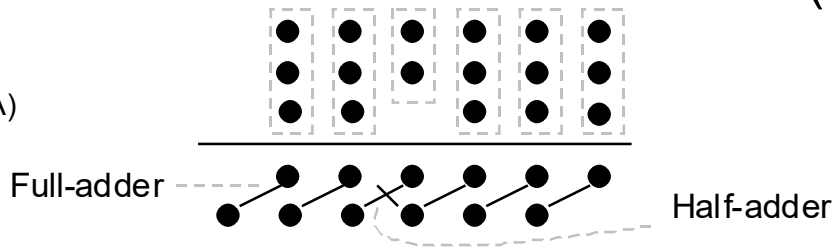




CSA
T: O(1)




Carry-propagate adder



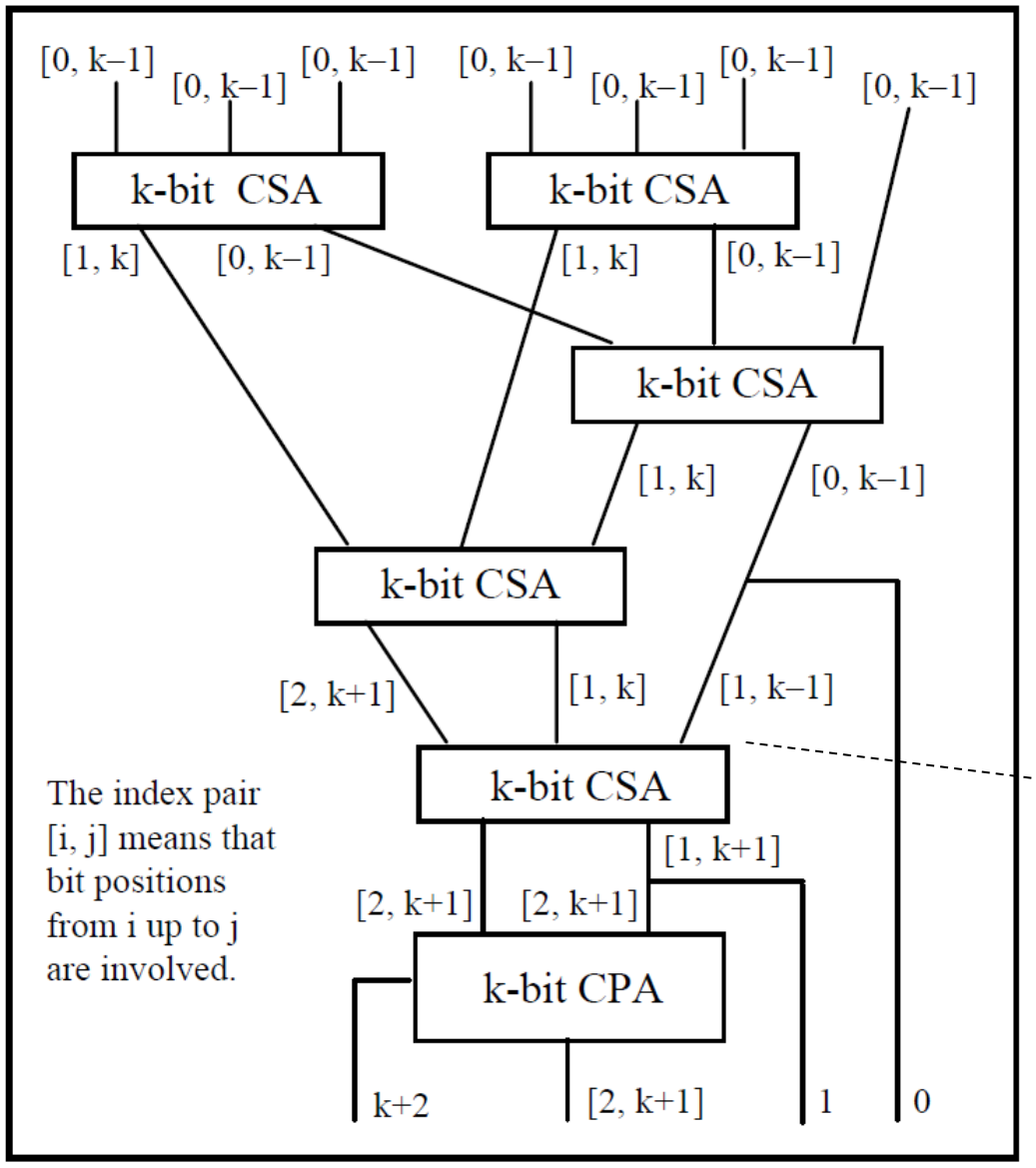




Carry-propagate adder (CPA) and carry-save adder (CSA) functions in dot notation.

Specifying full- and half-adder blocks, with their inputs and outputs, in dot notation.

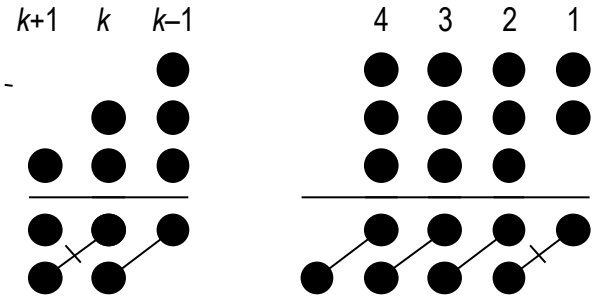
```
cp.
wire [k-1:0] a, b, c, sum;  // k-bit operands
wire  carry;  // 1-bit carry out
assign {carry, sum}=a+b+c; // generate one output of k+1 bits
```

# Bit-Width of Adders in a CSA Tree




Adding seven *k*-bit numbers and the CSA/CPA widths required.

Due to the gradual retirement (dropping out) of some of the LSB result bits, CSA widths do not vary much as we go down the tree levels



# # of Levels in a CSA Tree for n operands

| # of operands | # of CSA levels |
|---|---|
| 3 | 1 |
| 4 | 2 |
| $5 \le n \le 6$ | 3 |
| $7 \le n \le 9$ | 4 |
| $10 \le n \le 13$ | 5 |
| $14 \le n \le 19$ | 6 |
| $20 \le n \le 28$ | 7 |
| $29 \le n \le 42$ | 8 |
| $43 \le n \le 63$ | 9 |

**# of CSA levels ~= $\log_{1.5}(n/2)$ if (3:2) counters are used in each CSA level**

$$n' \ (2/3)^x = 2$$

$$n/2 = (3/2)^x$$

$$\log(n/2) = x' \ \log(3/2)$$

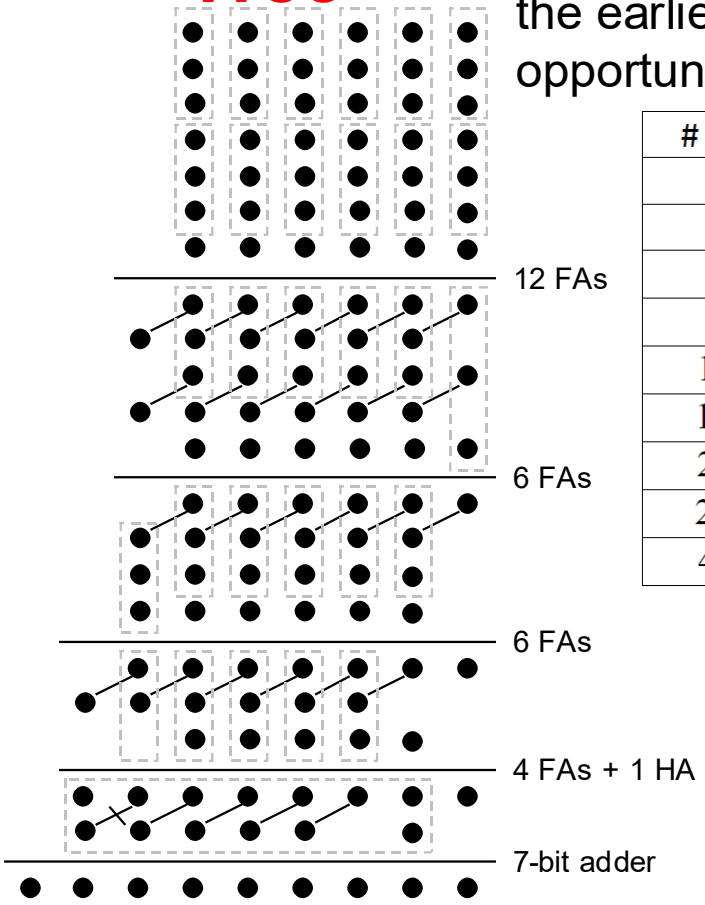
$$x = \frac{\log(n/2)}{\log(3/2)} = \log_{3/2}(n/2)$$

Example: n=12, 5 levels of CSA tree - delay of $5T_{CSA}$ instead of $10T_{CSA}$ in a linear cascade of 10 CSAs

# Wallace-like Tree

Wallace tree: Reduce the number of operands at the earliest possible opportunity

| # of operands | # of CSA levels |
|---|---|
| 3 | 1 |
| 4 | 2 |
| $5 \le n \le 6$ | 3 |
| $7 \le n \le 9$ | 4 |
| $10 \le n \le 13$ | 5 |
| $14 \le n \le 19$ | 6 |
| $20 \le n \le 28$ | 7 |
| $29 \le n \le 42$ | 8 |
| $43 \le n \le 63$ | 9 |

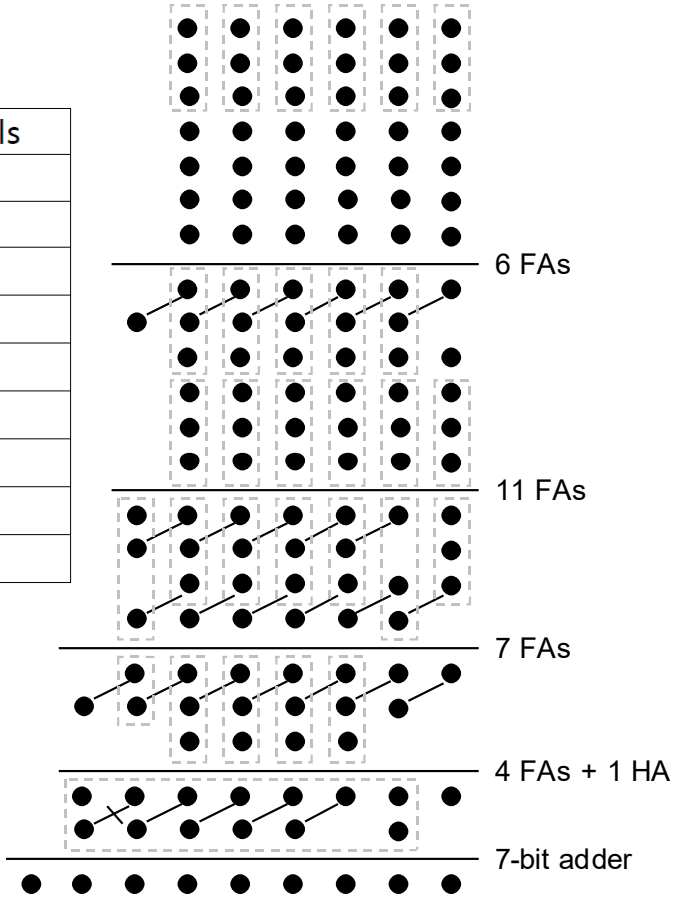



Total cost = 7-bit adder + 28 FAs + 1 HA

Addition of seven 6-bit numbers in dot notation.

Dadda tree: Postpone the reduction to the extent possible without causing added delay

# Dadda-like Tree




Total cost = 7-bit adder + 28 FAs + 1 HA

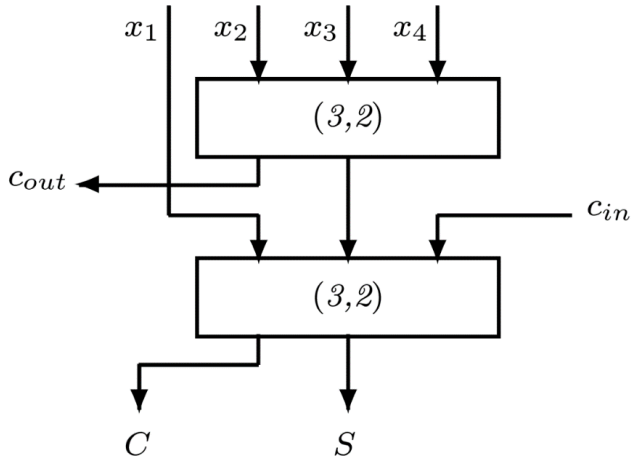
Adding seven 6-bit numbers using Dadda's strategy.

# Compression in Wallace vs. Dadda Tree

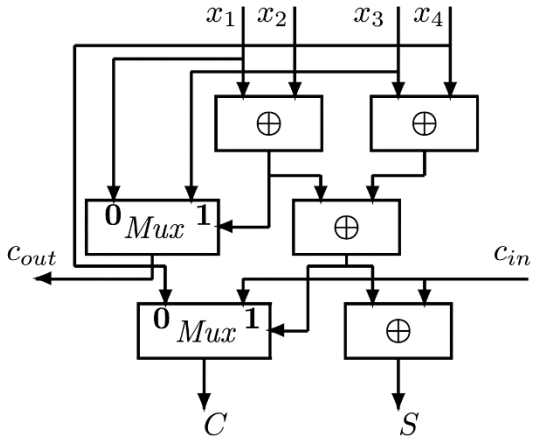
- Wallace tree
  - put every 3 rows (3 operands) into a group
    - ✓leave unchanged the remaining rows after grouping
  - use CSA to compress the group into 2 rows
    - ✓might have some variation near the LSB and MSB boundaries
- Dadda tree
  - Partition into groups with 3 rows in each group
  - Reduce the height of a group from 3 to 2 via CSA until the total height is the upper bound of one less CSA level
- Another compression approach is column-based
  - **Perform reduction from the LSB column**
  - **Need to careful about the selection of inputs to a (3:2) counter / (2:2) counter (smallest delay first)**

Hou-Jen Ko and Shen-Fu Hsiao, "Design and Application of Faithfully Rounded and Truncated Multipliers with Combined Deletion, Reduction, Truncation, and Rounding," IEEE Trans. Circuits and Systems, Part-II, vol. 58, no. 5, pp. 304-308, May 2011

# (4;2) Compressors




- Same reduction rate of 2:1 achieved without SD representations by using (4;2) compressors
- Designed so that $c_{out}$ is not a function of $c_{in}$ to avoid a ripple-carry effect



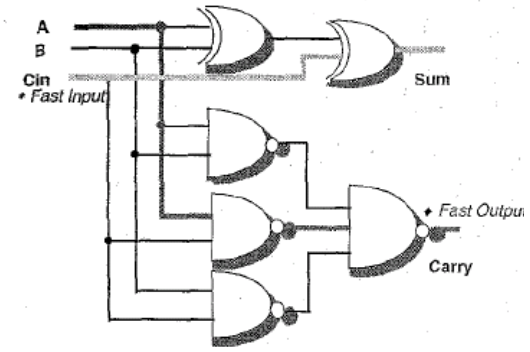

$$x_1 + x_2 + x_3 + x_4 + c_{in} = S + 2(C + c_{out})$$

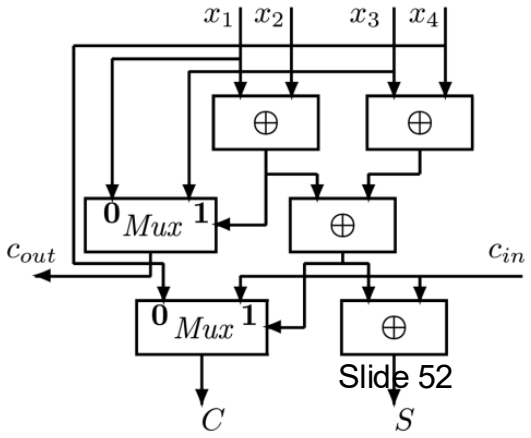
- (4;2) compressor may be implemented as a multi-level circuit with a smaller overall delay compared to implementation based on 2 (3,2) counters

# Comparing Delay of CSATrees

| Number of operands | Number of levels using (3,2) | Number of levels using (4;2) | Equivalent delay |
|---|---|---|---|
| 3 | 1 | 1 | 1.5 |
| 4 | 2 | 1 | 1.5 |
| 5 – 6 | 3 | 2 | 3 |
| 7 – 8 | 4 | 2 | 3 |
| 9 | 4 | 3 | 4.5 |
| 10 – 13 | 5 | 3 | 4.5 |
| 14 – 16 | 6 | 3 | 4.5 |
| 17 – 19 | 6 | 4 | 6 |
| 20 – 28 | 7 | 4 | 6 |
| 29 – 32 | 8 | 4 | 6 |
| 33 – 42 | 8 | 5 | 7.5 |

| Number of operands | Number of levels |
|---|---|
| 3 | 1 |
| 4 | 2 |
| $5 \leq k \leq 6$ | 3 |
| $7 \leq k \leq 9$ | 4 |
| $10 \leq k \leq 13$ | 5 |
| $14 \leq k \leq 19$ | 6 |
| $20 \leq k \leq 28$ | 7 |
| $29 \leq k \leq 42$ | 8 |
| $43 \leq k \leq 63$ | 9 |



# Carry-Lookahead Adders (CLA)

# 4-bit CLA

- bit-level carry “generate” signals
- bit-level carry “propagate” signals

$$g_i = a_i b_i$$

$$p_i = a_i \oplus b_i$$

$$(\text{or } p_i = a_i + b_i \text{ in } c_{i+1} = g_i + p_i c_i \ )$$

- carry-out=1 if (generate=1) or (propagate=1 and carry-in=1)

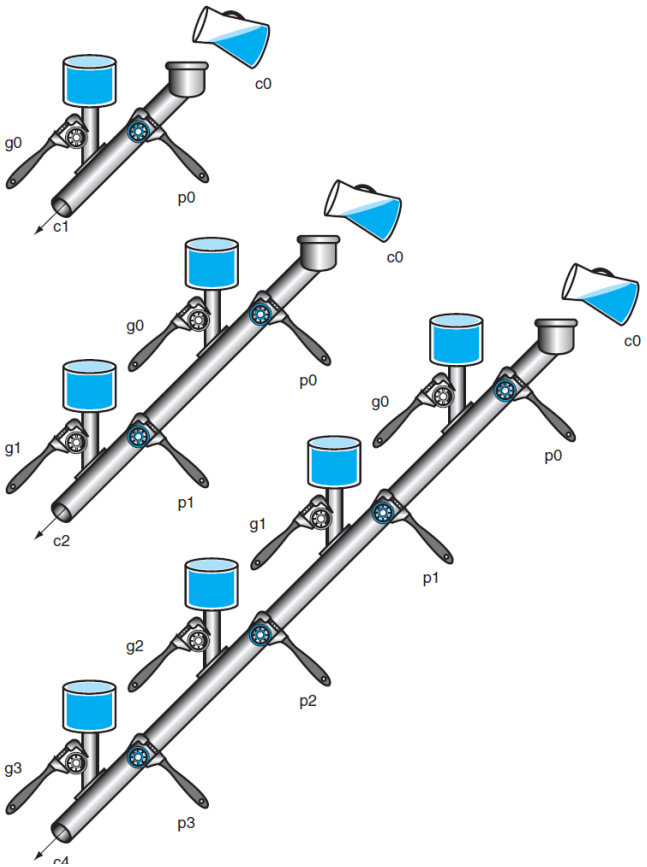
$$c_{i+1} = g_i + p_i c_i$$

$$c1 = g0 + (p0 \cdot c0)$$

$$c2 = g1 + (p1 \cdot g0) + (p1 \cdot p0 \cdot c0)$$

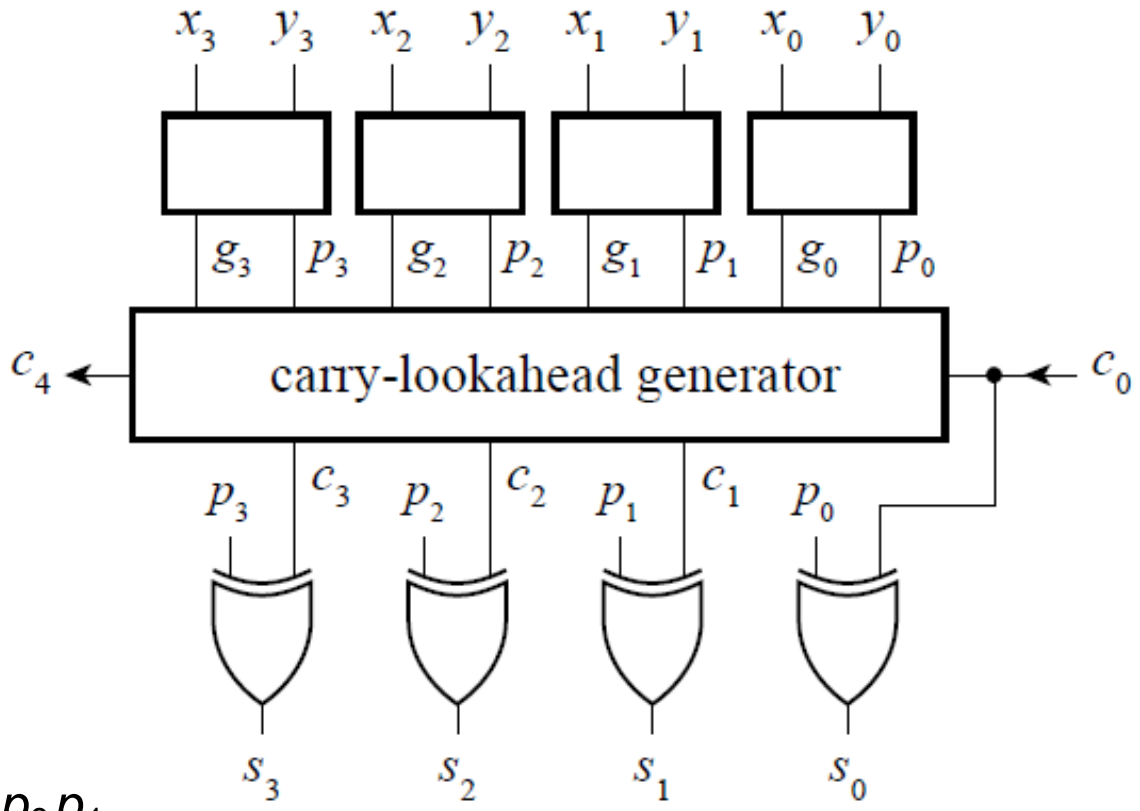
$$c3 = g2 + (p2 \cdot g1) + (p2 \cdot p1 \cdot g0) + (p2 \cdot p1 \cdot p0 \cdot c0)$$

$$c4 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0) + (p3 \cdot p2 \cdot p1 \cdot p0 \cdot c0)$$



# 4-bit carry-look-ahead (CLA)

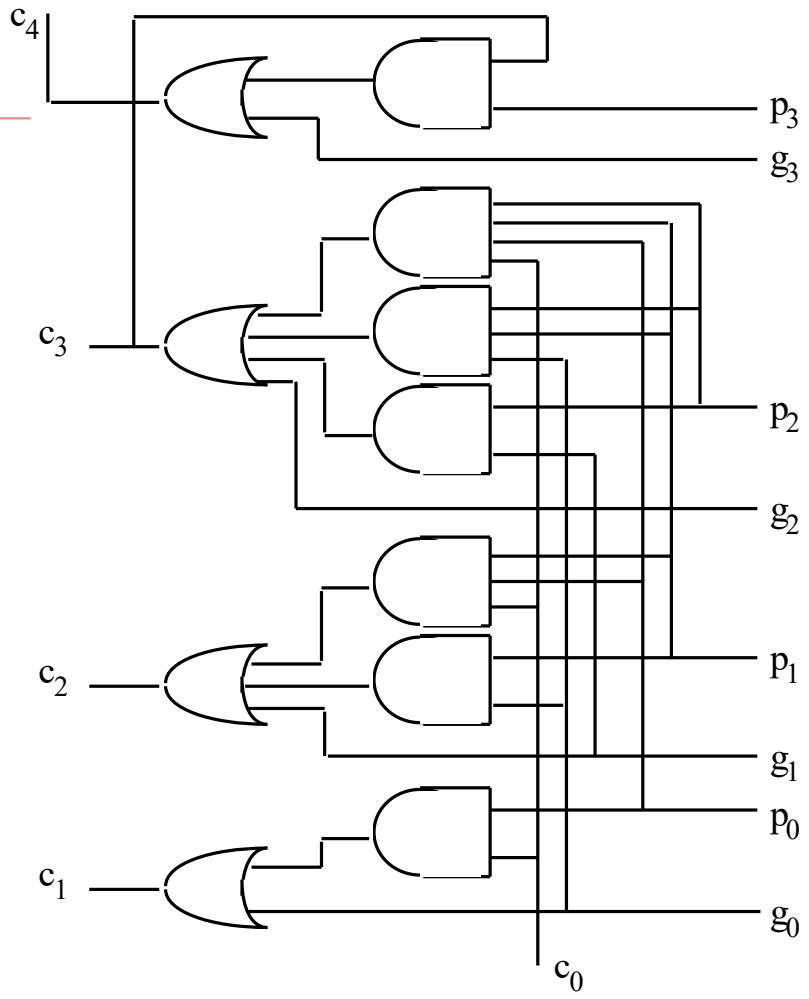
- calculate the internal carry bits based on
  - bit-level propagate and generate signals
  - the block carry-in bit




$$c_1 = g_0 \vee c_0 p_0$$
$$c_2 = g_1 \vee g_0 p_1 \vee c_0 p_0 p_1$$
$$c_3 = g_2 \vee g_1 p_2 \vee g_0 p_1 p_2 \vee c_0 p_0 p_1 p_2$$
$$c_4 = g_3 \vee g_2 p_3 \vee g_1 p_2 p_3 \vee g_0 p_1 p_2 p_3 \vee c_0 p_0 p_1 p_2 p_3$$

# Four-Bit Carry-Lookahead Adder

Complexity reduced by deriving the carry-out indirectly




Full carry lookahead is quite practical for a 4-bit adder

$c_1 = g_0 \vee c_0 p_0$

$c_2 = g_1 \vee g_0 p_1 \vee c_0 p_0 p_1$

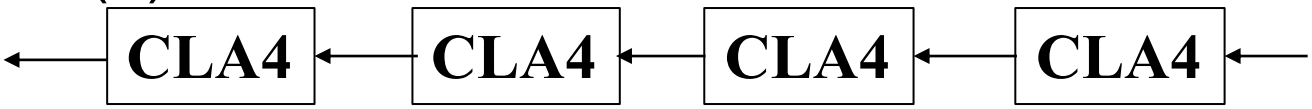
$c_3 = g_2 \vee g_1 p_2 \vee g_0 p_1 p_2 \vee c_0 p_0 p_1 p_2$

$c_4 = g_3 \vee g_2 p_3 \vee g_1 p_2 p_3 \vee g_0 p_1 p_2 p_3 \vee c_0 p_0 p_1 p_2 p_3$

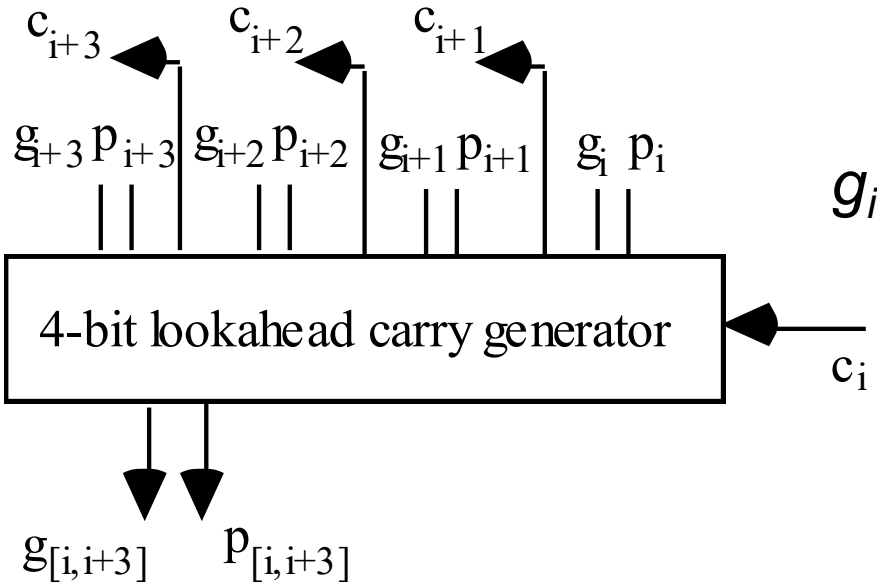
$= g_3 \vee c_3 p_3$

# Cascade of CLA4 and hierarchical CLA

- delay is reduced from n to n/4 for n-bit adders
  - but still order O(n)




- want $O(\log_4 n)$ delay → hierarchical CLA
- repeat the concept of p[i], g[i] (for single bit position i) to a larger block p[i,i+3], g[i,i+3] (for 4-bit block)




$$g_{[i,i+3]} = g_{i+3} \vee g_{i+2} p_{i+3} \vee g_{i+1} p_{i+2} p_{i+3} \vee g_i p_{i+1} p_{i+2} p_{i+3}$$

$$p_{[i,i+3]} = p_i p_{i+1} p_{i+2} p_{i+3}$$

# Block (Group) Generate and Propagate

**Block (Group) *generate*** and ***propagate*** signals for CLA4

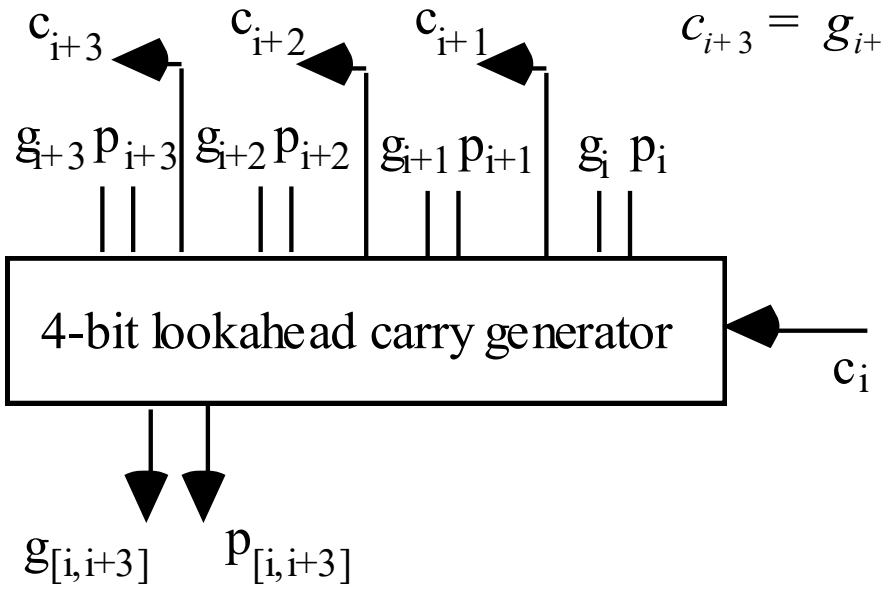
$$g_{[i,i+3]} = g_{i+3} \vee g_{i+2}p_{i+3} \vee g_{i+1}p_{i+2}p_{i+3} \vee g_i\, p_{i+1}p_{i+2}p_{i+3}$$

$$p_{[i,i+3]} = p_i\, p_{i+1}p_{i+2}p_{i+3}$$

$$c_{i+1} = g_i + p_i c_i$$

$$c_{i+2} = g_{i+1} + p_{i+1}g_i + p_{i+1}p_i c_i$$

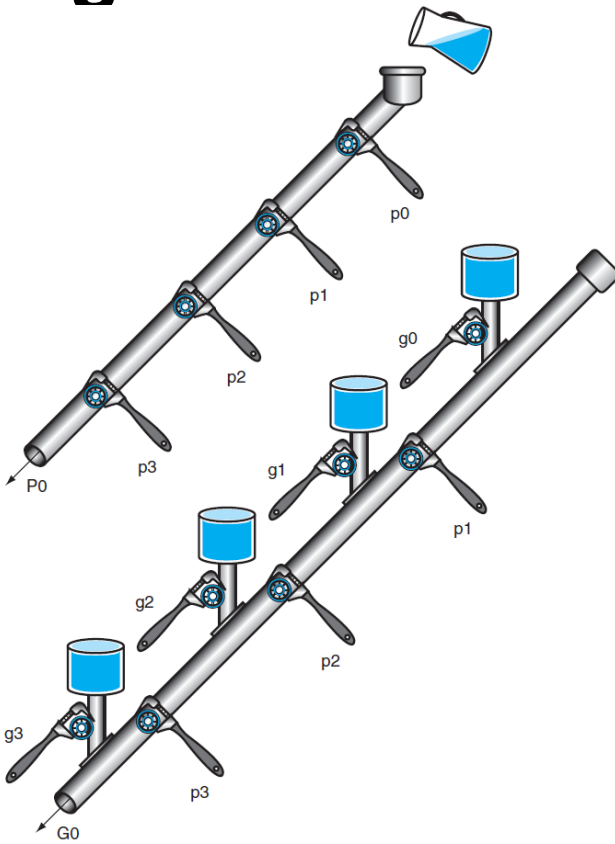
$$c_{i+3} = g_{i+2} + p_{i+2}g_{i+1} + p_{i+2}p_{i+1}g_i + p_{i+2}p_{i+1}p_i c_i$$




Schematic diagram of a 4-bit lookahead carry generator.

# Group (Block) P and G Signals

- group-level “generate”, “propagate’
- E.g. a group has 4 bits




$$P0 = p3 \cdot p2 \cdot p1 \cdot p0$$

$$P1 = p7 \cdot p6 \cdot p5 \cdot p4$$

$$P2 = p11 \cdot p10 \cdot p9 \cdot p8$$

$$P3 = p15 \cdot p14 \cdot p13 \cdot p12$$

$$G0 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0)$$

$$G1 = g7 + (p7 \cdot g6) + (p7 \cdot p6 \cdot g5) + (p7 \cdot p6 \cdot p5 \cdot g4)$$

$$G2 = g11 + (p11 \cdot g10) + (p11 \cdot p10 \cdot g9) + (p11 \cdot p10 \cdot p9 \cdot g8)$$

$$G3 = g15 + (p15 \cdot g14) + (p15 \cdot p14 \cdot g13) + (p15 \cdot p14 \cdot p13 \cdot g12)$$

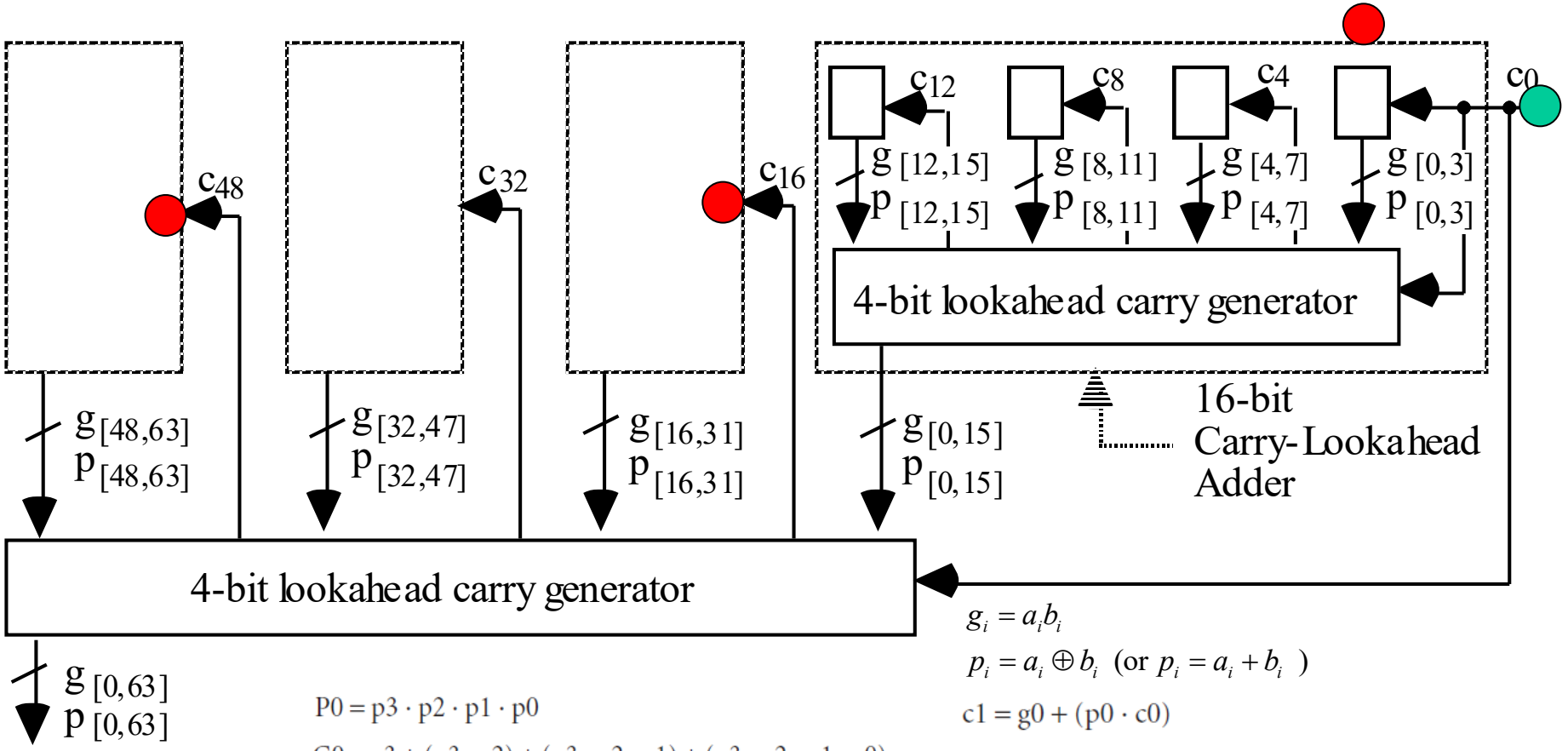
$$C1 = G0 + (P0 \cdot c0)$$

$$C2 = G1 + (P1 \cdot G0) + (P1 \cdot P0 \cdot c0)$$

$$C3 = G2 + (P2 \cdot G1) + (P2 \cdot P1 \cdot G0) + (P2 \cdot P1 \cdot P0 \cdot c0)$$

$$C4 = G3 + (P3 \cdot G2) + (P3 \cdot P2 \cdot G1) + (P3 \cdot P2 \cdot P1 \cdot G0) + (P3 \cdot P2 \cdot P1 \cdot P0 \cdot c0)$$

# A 64-bit Hierarchical CLA




$$g_i = a_i b_i$$

$$p_i = a_i \oplus b_i \quad (\text{or } p_i = a_i + b_i \ )$$

$$P0 = p3 \cdot p2 \cdot p1 \cdot p0$$

$$G0 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0)$$

$$C1 = G0 + (P0 \cdot c0)$$

$$C2 = G1 + (P1 \cdot G0) + (P1 \cdot P0 \cdot c0)$$

$$C3 = G2 + (P2 \cdot G1) + (P2 \cdot P1 \cdot G0) + (P2 \cdot P1 \cdot P0 \cdot c0)$$

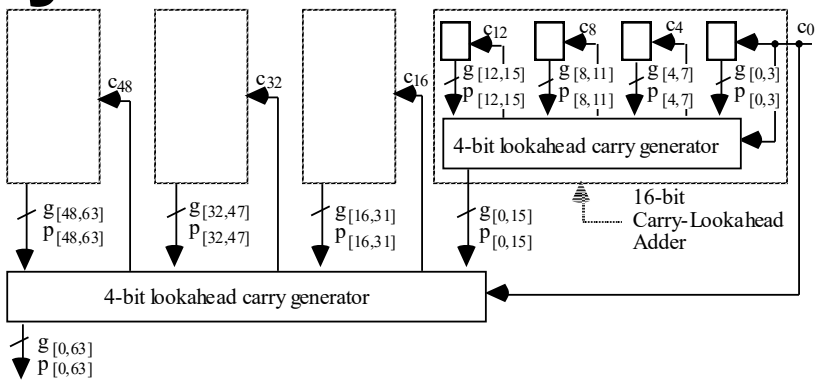
$$c1 = g0 + (p0 \cdot c0)$$

$$c2 = g1 + (p1 \cdot g0) + (p1 \cdot p0 \cdot c0)$$

$$c3 = g2 + (p2 \cdot g1) + (p2 \cdot p1 \cdot g0) + (p2 \cdot p1 \cdot p0 \cdot c0)$$

Building a 64-bit carry-lookahead adder from 16 4-bit adders and 5 lookahead carry generators.

# critical path delay of CLA-64




- sum of the following delays
  - T(p,g) = Txor = max(Txor, Tand)
    - ✓ delay for p, g of individual bits
  - T(P4,G4) = TPG
    - ✓ 1st level CLA delay for 4-bit block P, G
  - T(P16,G16) = TPG
    - ✓ 2nd level CLA delay for 16-bit block P, G
  - T(c0->c48) = TC3
    - ✓ 3rd level CLA delay for c0 -> c48
  - T(c48->c60) = TC3
    - ✓ 2nd level CLA delay for c48 -> c60
  - T(c60->c63) = TC3
    - ✓ 1st level CLA dealy for c60 -> c63
  - T(s63) = Txor
    - ✓ full adder delay from carry-in bit ci to sum bit si = ci xor pi

$$P0 = p3 \cdot p2 \cdot p1 \cdot p0$$

$$G0 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0)$$

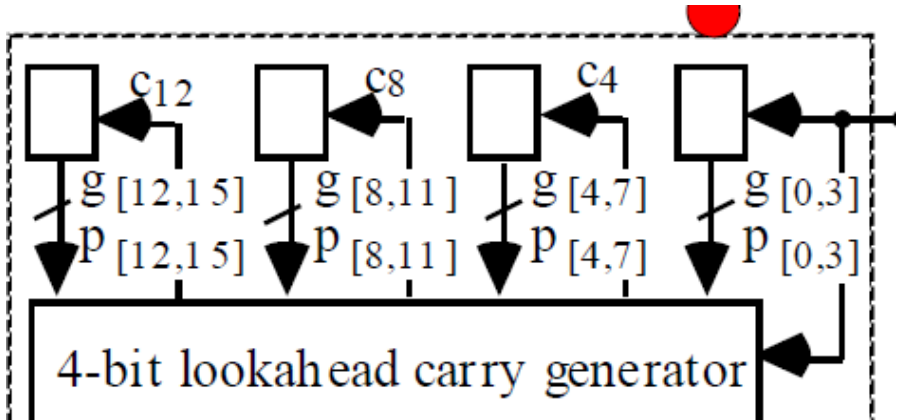
$$C1 = G0 + (P0 \cdot c0)$$

$$C2 = G1 + (P1 \cdot G0) + (P1 \cdot P0 \cdot c0)$$

$$C3 = G2 + (P2 \cdot G1) + (P2 \cdot P1 \cdot G0) + (P2 \cdot P1 \cdot P0 \cdot c0)$$

# CLA-4 and CLA-16

- $p_{[i:i+3]} = p_{i+3}p_{i+2}p_{i+1}p_i$
- $g_{[i:i+3]} = g_{i+3} + g_{i+2}p_{i+3} + g_{i+1}p_{i+2}p_{i+3} + g_ip_{i+1}p_{i+2}p_{i+3}$
- similar for $p_{[i+4:i+7]}$, $g_{[i+4:i+7]}$), $p_{[i+8:i+11]}$, $g_{[i+8:i+11]}$), and ($p_{[i+12:i+15]}$, $g_{[i+12:i+15]}$)
- internal carry-out $c_{i+4}$, $c_{i+8}$, $c_{i+12}$ from CLA-16 block
- $c_{i+4} = g_{[i:i+3]} + p_{[i:i+3]}c_i$
- $c_{i+8} = g_{[i+4:i+7]} + g_{[i:i+3]}p_{[i+4:i+8]} + c_ig_{[i:i+3]}p_{[i+4:i+8]}$
- $c_{i+12} = g_{[i+8:i+12]} + g_{[+4:i+7]}p_{[i+8:i+12}+ g_{[:i+3]}p_{[i+8:i+12]}p_{[i+4:i+7]]} + c_ip_{[:i+3]}p_{[i+4:i+7]}p_{[i+8:i+12]}$



# Critical path delay of 3-level CLA64

- Delay from top level (1st level) down to the 3rd level, and then up to the 1st level
- $T_{total}$ =

  $T_{xor}$

  + $T_{pg4}$ (level 1 block pg)
  + $T_{pg4}$ (level2 block pg)
  + $T_c$ (level 3 internal carry)
  + $T_c$ (level 2 internal carry)
  + $T_c$ (level 1 internal carry)
  + $T_{xor}$ (sum)
- quiz
  - latency of CLA32 ?
  - level 3: $T_c$ -> $T_{and}+T_{or}$

$$P = p_{i+3}p_{i+2}p_{i+1}p_i$$

$$G = g_{i+3} + p_{i+3}g_{i+2} + p_{i+3}p_{i+2}g_{i+1} + p_{i+3}p_{i+2}p_{i+1}g_i$$

$$c_{i+3} = g_{i+3} + p_{i+3}g_{i+2} + p_{i+3}p_{i+2}g_{i+1} + p_{i+3}p_{i+2}p_{i+1}c_i$$

